# LAPORAN PROYEK UJIAN TENGAH SEMESTER
# KLASIFIKASI HURUF BAHASA ISYARAT AMERIKA (ASL) PADA DATASET SIGN LANGUAGE MNIST MENGGUNAKAN DEEP NEURAL NETWORK DAN CONVOLUTIONAL NEURAL NETWORK

Mata Kuliah: IF2121 – Pembelajaran Mendalam
Semester: Ganjil
TA. 2026-2027

**Nama Kelompok: Kelompok1**
**NIM & Nama Anggota Kelompok:**
231111408 – Steven David H Malau
231111488 – Wan Saka Nasa
231111207 – Edwin Valentino Tarigan
231110581 – Eric Chandra
231111650 – Dhaifan Al Ghani

**[isi manual]** sisipkan logo Universitas Mikroskil di sini

**PROGRAM STUDI S-1 TEKNIK INFORMATIKA**
**FAKULTAS INFORMATIKA**
**UNIVERSITAS MIKROSKIL**

# DAFTAR ISI

DAFTAR ISI....2
DAFTAR GAMBAR....4
DAFTAR TABEL....6
PEMBAGIAN TUGAS....7
BAGIAN A. LAPORAN TEKNIS PROYEK....8
1. Kompleksitas Masalah....8
a. Mengapa masalah ini dianggap kompleks?....8
b. Apa tantangan utama dalam menyelesaikan masalah ini?....8
c. Pengembangan dan penyempurnaan dari proyek sebelumnya....9
2. Dataset yang Digunakan....9
a. Sumber dataset....9
b. Jumlah data, kelas/label, format, dan ukuran data....9
c. Apakah dataset bersifat real-world atau sintetis?....10
d. Seberapa kompleks dataset tersebut?....10
3. Exploratory Data Analysis (EDA)....10
a. Distribusi label/kelas....10
b. Word count....10
c. Histogram fitur numerik....10
d. Korelasi antarfitur....11
e. Contoh sampel data per kelas....11
f. Temuan penting dari EDA yang memengaruhi preprocessing....12
4. Preprocessing Data....13
a. Pembersihan data....13
b. Normalisasi dan rescale piksel....13
c. Augmentasi data....13
d. Penanganan missing value, outlier, dan ketidakseimbangan kelas....14
e. Pembagian data dan pencegahan kebocoran data....14
5. Model yang Digunakan....15

a. Jenis model.....15
b. Arsitektur model.....15
c. Konfigurasi pelatihan.....16
d. Alasan pemilihan model dan arsitektur.....17
e. Dua fitur unik.....17
6. Hasil Evaluasi.....20
a. Metrik evaluasi yang digunakan dan alasannya.....20
b. Nilai metrik pada data testing.....20
c. Grafik loss dan akurasi data training dan validation.....21
d. Perbandingan antarskenario.....22
e. Analisis hasil.....25
7. Teknologi yang Digunakan.....27
a. Bahasa pemrograman.....27
b. Library dan versinya.....27
c. Lingkungan pengembangan.....28
d. Framework aplikasi dan platform hosting.....28
8. Deployment dan Demo.....28
a. Link deployment.....28
b. Tahapan deployment.....28
c. Penjelasan video demo dan deployment.....30
9. Lampiran.....33
a. Cuplikan kode penting.....33
b. Grafik evaluasi.....34
10. Daftar Pustaka.....36
BAGIAN B. TAUTAN.....38

# DAFTAR GAMBAR

Gambar 1 - *Distribusi kelas pada data latih dan uji resmi*.....10
Gambar 2 - *Histogram intensitas piksel dan kecerahan rata-rata citra latih vs uji*.....11
Gambar 3 - *Korelasi antarpiksel pada baris ke-14 dan kolom ke-14*.....11
Gambar 4 - *Contoh citra untuk setiap huruf*.....12
Gambar 5 - *Citra rata-rata setiap kelas*.....12
Gambar 6 - *Citra asli (baris pertama) dan tiga variasi hasil augmentasi*.....14
Gambar 7 - *Diagram arsitektur CNN akhir (E5)*.....15
Gambar 8 - *Trade-off akurasi–coverage dan distribusi confidence (dalam vs luar domain)*.....19
Gambar 9 - *Contoh input di luar domain dan keputusan model pada τ = 0,5 (merah = tidak tertolak)*.....19
Gambar 10 - *Ukuran berkas dan akurasi uji model Keras vs varian TFLite*.....20
Gambar 11 - *Kurva loss, akurasi, dan learning rate (latih vs validasi) model E5*.....22
Gambar 12 - *Kurva loss dan akurasi (latih vs validasi) DNN baseline E1*.....22
Gambar 13 - *Perbandingan performa skenario E1–E5 dan kurva akurasi validasi*.....22
Gambar 14 - *Loss latih dan akurasi validasi per fungsi aktivasi*.....23
Gambar 15 - *Loss latih dan akurasi validasi per optimizer*.....24
Gambar 16 - *Kesalahan perceptron, loss neuron sigmoid, dan kurva MLP manual*.....24
Gambar 17 - *Akurasi proyek sebelumnya (abu-abu) vs proyek ini (biru)*.....25
Gambar 18 - *Confusion matrix 24×24 model E5 pada data uji*.....26
Gambar 19 - *Akurasi (recall) per huruf model E5*.....26
Gambar 20 - *Grad-CAM model E5 pada enam citra uji (semua prediksi benar)*.....27
Gambar 21 - *Tangkapan layar langkah 1: menyimpan model*.....28
Gambar 22 - *Tangkapan layar langkah 2: menyiapkan skrip inferensi*.....29
Gambar 23 - *Tangkapan layar langkah 3: membangun antarmuka*.....29
Gambar 24 - *Tangkapan layar langkah 4: mencatat dependensi*.....29
Gambar 25 - *Tangkapan layar langkah 5: mengunggah ke platform hosting*.....30
Gambar 26 - *Tangkapan layar langkah 6: menguji aplikasi*.....30
Gambar 27 - *Tampilan antarmuka aplikasi di Hugging Face Spaces*.....31

Gambar 28 - *Contoh input dan output aplikasi (hijau = benar, merah = salah)*.....31
Gambar 29 - *Tangkapan layar aplikasi: input derau diterima sebagai huruf P (contoh prediksi salah)*.....32
Gambar 30 - *Kurva loss dan akurasi (latih vs validasi) DNN teregularisasi E2*.....34
Gambar 31 - *Kurva loss dan akurasi (latih vs validasi) CNN baseline E3*.....34
Gambar 32 - *Kurva loss dan akurasi (latih vs validasi) CNN teregularisasi E4*.....35
Gambar 33 - *Distribusi confidence prediksi model E5 pada data uji (seluruhnya benar)*.....35
Gambar 34 - *Strategi learning rate scheduling (konsep)*.....35
Gambar 35 - *Representasi tensor: citra (28×28×1), reduce_mean, dan reshape ke (B, 784)*.....35

# DAFTAR TABEL

Tabel 1 - *Pembagian tugas anggota kelompok*.....7
Tabel 2 - *Pengembangan dan penyempurnaan dari proyek sebelumnya*.....9
Tabel 3 - *Ringkasan dataset Sign Language MNIST*.....9
Tabel 4 - *Temuan EDA dan dampaknya pada preprocessing*.....12
Tabel 5 - *Jenis dan parameter augmentasi data*.....13
Tabel 6 - *Pembagian data*.....14
Tabel 7 - *Rincian lapisan CNN akhir (B = ukuran batch)*.....15
Tabel 8 - *Ringkasan arsitektur skenario E1–E5*.....16
Tabel 9 - *Konfigurasi pelatihan (ReduceLR = ReduceLROnPlateau: faktor 0,5, patience 3, min_lr 1e-5)*.....16
Tabel 10 - *Dampak fitur unik 1 pada data uji*.....18
Tabel 11 - *Kemampuan menolak input di luar domain (tanpa fitur ini, 0% ditolak)*.....18
Tabel 12 - *Dampak fitur unik 2: ukuran, akurasi, latensi (★ = varian deployment)*.....19
Tabel 13 - *Nilai metrik model akhir pada data uji*.....20
Tabel 14 - *Classification report per huruf (E5)*.....21
Tabel 15 - *Hasil seluruh skenario*.....22
Tabel 16 - *Perbandingan fungsi aktivasi (Adam, 15 epoch)*.....23
Tabel 17 - *Perbandingan optimizer*.....23
Tabel 18 - *Ringkasan perbandingan tanpa dan dengan fitur unik*.....24
Tabel 19 - *Library yang digunakan*.....27
Tabel 20 - *Link deployment*.....28
Tabel 21 - *Ringkasan contoh input dan output aplikasi*.....32
Tabel 22 - *Pembagian segmen video (setiap anggota menjelaskan bagiannya)*.....32

# PEMBAGIAN TUGAS

Tabel 1 merinci peran dan pekerjaan setiap anggota. Setiap anggota memegang satu tahapan utama proyek (data, model, optimasi, evaluasi dan fitur unik, serta deployment) dengan beban yang seimbang (±20% per anggota), dan setiap anggota menjelaskan bagiannya sendiri dalam video demo.

*Tabel 1 - Pembagian tugas anggota kelompok*

| NIM | Nama | Peran | Rincian pekerjaan |
|---|---|---|---|
| 231111408 | Steven David H Malau | Ketua Tim & Model Engineer (CNN) | • Koordinasi tim dan konsultasi topik ke dosen<br>• Merancang arsitektur CNN (E3–E5): konvolusi, pooling, BatchNorm, Dropout<br>• Melatih dan melakukan tuning model CNN akhir<br>• Integrasi notebook akhir dan review laporan |
| 231111488 | Wan Saka Nasa | Data Engineer (EDA, Preprocessing, tf.data) | • Akuisisi dataset (KaggleHub) dan pengecekan integritas<br>• EDA: distribusi kelas, histogram piksel, korelasi piksel, contoh per kelas<br>• Pembersihan, normalisasi, reshape, dan pembagian data tanpa kebocoran<br>• Representasi tensor, pipeline tf.data, dan augmentasi data |
| 231111207 | Edwin Valentino Tarigan | DNN & Optimasi (Perceptron, MLP, Optimizer) | • Perceptron Rosenblatt, neuron sigmoid, dan MLP manual (GradientTape)<br>• Model DNN/MLP E1–E2<br>• Studi fungsi aktivasi dan optimizer (SGD, Momentum, RMSProp, Adam)<br>• Callback: EarlyStopping, ReduceLROnPlateau, Checkpoint, TensorBoard |
| 231110581 | Eric Chandra | Evaluasi & Fitur Unik (Threshold/OOD, TFLite, Grad-CAM) | • Evaluasi: metrik, classification report, confusion matrix, analisis hasil<br>• Fitur unik 1: confidence threshold & deteksi input di luar domain<br>• Fitur unik 2: kompresi model ke TensorFlow Lite<br>• Interpretasi Grad-CAM |
| 231111650 | Dhaifan Al Ghani | Deployment & Dokumentasi (Gradio, HF Spaces, GitHub, Video) | • Skrip inferensi dan antarmuka Gradio<br>• Deployment ke Hugging Face Spaces dan pengujian aplikasi<br>• Repositori GitHub dan README<br>• Video demo, penyusunan laporan, dan pengumpulan |

# BAGIAN A. LAPORAN TEKNIS PROYEK

## 1. Kompleksitas Masalah

Proyek ini membangun model yang mengenali 24 huruf statis American Sign Language (ASL) dari citra tangan grayscale berukuran 28×28 piksel. Pengenalan huruf isyarat (fingerspelling) adalah fondasi sistem penerjemah bahasa isyarat yang dapat membantu komunikasi penyandang tunarungu. Menurut WHO, lebih dari 430 juta orang hidup dengan gangguan pendengaran yang memerlukan rehabilitasi (World Health Organization, 2021). Sesuai ketentuan tugas, model dibatasi pada Deep Neural Network (DNN/MLP) dan Convolutional Neural Network (CNN).

### a. Mengapa masalah ini dianggap kompleks?

1. **Kemiripan antarkelas sangat tinggi (fine-grained).** Huruf berbasis kepalan (A, E, M, N, S, T) hanya berbeda pada posisi ibu jari, dan huruf dua jari (U, V, R, K) hanya berbeda pada jarak atau silangan jari. Pada proyek sebelumnya, pasangan M→N, D→U, dan S→M termasuk yang paling sering tertukar.
2. **Resolusi rendah dan dimensi tinggi.** Citra hanya 28×28 piksel grayscale sehingga banyak detail jari yang hilang. Bagi DNN, setiap citra adalah vektor 784 dimensi tanpa informasi spasial.
3. **Variasi kondisi citra.** Kecerahan antarcitra sangat bervariasi (simpangan baku piksel ±48,8 pada skala 0–255), begitu pula posisi, rotasi, skala, dan latar tangan.
4. **Pergeseran distribusi data latih dan uji.** Data latih dibentuk dengan memperbanyak sejumlah kecil foto asli, sehingga validasi dari data latih sangat optimistis. Contohnya, DNN E2 mencapai 100,00% pada validasi tetapi hanya 82,18% pada data uji.
5. **Ketidakseimbangan kelas pada data uji.** Rasio kelas terbanyak (E, 498 citra) terhadap tersedikit (R, 144 citra) pada data uji mencapai 3,46.
6. **Input dunia nyata yang tidak terduga.** Pada aplikasi, pengguna dapat mengunggah foto apa saja, padahal softmax selalu memaksakan salah satu dari 24 huruf.

### b. Apa tantangan utama dalam menyelesaikan masalah ini?

1. Membuat model yang menggeneralisasi ke data uji berdistribusi berbeda, bukan sekadar menghafal data latih yang mirip satu sama lain.
2. Membedakan pasangan huruf yang secara visual hampir identik pada resolusi 28×28.
3. Mencegah kebocoran data: data uji tidak boleh dipakai untuk early stopping, pemilihan model, atau penentuan ambang.
4. Menolak secara andal input yang berada di luar domain, alih-alih memberi jawaban salah dengan sangat yakin.
5. Menghasilkan model yang ringan dan cepat agar dapat di-deploy sebagai aplikasi web.

### c. Pengembangan dan penyempurnaan dari proyek sebelumnya

Tabel 2 merangkum bagian yang ditambah, diperbaiki, atau diganti berdasarkan audit terhadap notebook dan laporan proyek sebelumnya.

*Tabel 2 - Pengembangan dan penyempurnaan dari proyek sebelumnya*

| Aspek | Proyek sebelumnya | Proyek ini | Jenis |
|---|---|---|---|
| Jumlah data | Subset 1.200 citra (50/kelas) | Seluruh data: 27.455 latih + 7.172 uji resmi | Diperbaiki |
| Pembagian data | Data uji juga dipakai sebagai validasi (kebocoran data) | Latih/validasi stratified 90/10; data uji resmi terpisah | Diperbaiki |
| Cakupan metode | Campuran ML klasik (DT, KNN, LR, K-Means) | Dibatasi pada DNN dan CNN sesuai ketentuan | Diganti |
| Input pipeline | Array NumPy langsung | tf.data: cache, shuffle, augmentasi, batch, prefetch | Ditambah |
| DNN & optimasi | Tidak ada MLP dan studi optimizer | Perceptron, MLP manual & Keras, studi aktivasi & optimizer, LR scheduling | Ditambah |
| Augmentasi | Tidak ada | Rotasi, zoom, translasi, brightness, contrast | Ditambah |
| Fitur unik | Tidak ada | Confidence threshold + deteksi OOD; kompresi TFLite | Ditambah |
| Interpretasi | Tidak ada | Grad-CAM | Ditambah |
| Deployment | Tidak ada | Aplikasi Gradio di Hugging Face Spaces + repositori GitHub | Ditambah |
| Hasil CNN | 71,67% pada 240 citra (laporan menulis 93,33%) | 100,00% pada 7.172 citra uji resmi | Diperbaiki |
| Konsistensi laporan | Placeholder [ISI] dan angka tidak cocok dengan output | Semua angka diambil dari results.json hasil running | Diperbaiki |

## 2. Dataset yang Digunakan

### a. Sumber dataset

Dataset yang digunakan adalah Sign Language MNIST (Datamunge, 2017) dari Kaggle: https://www.kaggle.com/datasets/datamunge/sign-language-mnist. Dataset ini dirancang sebagai pengganti MNIST (LeCun et al., 1998) untuk pengenalan gestur tangan. Pada running ini, dataset diunduh otomatis melalui KaggleHub (datamunge/sign-language-mnist).

### b. Jumlah data, kelas/label, format, dan ukuran data

*Tabel 3 - Ringkasan dataset Sign Language MNIST*

| Atribut | Keterangan |
|---|---|
| Jumlah data | 34.627 citra: 27.455 latih + 7.172 uji resmi |
| Kelas/label | 24 kelas: huruf A–Y tanpa J dan Z (keduanya memerlukan gerakan); label asli 0–24 tanpa 9 |
| Format | CSV; setiap baris = 1 kolom label + 784 kolom piksel (row-major) |
| Ukuran citra | 28×28 piksel, grayscale 1 kanal, intensitas 0–255 |
| Data latih per kelas | 957–1.294 citra |
| Data uji per kelas | 144–498 citra |

### c. Apakah dataset bersifat real-world atau sintetis?

Dataset ini bersifat real-world yang diperbanyak secara sintetis. Citra dasarnya adalah foto tangan sungguhan dari beberapa pengguna dengan latar berbeda. Menurut deskripsi di Kaggle, foto tersebut dipotong, diubah ke grayscale, diperkecil, lalu diperbanyak menjadi banyak variasi melalui pipeline pengolahan citra (filter, pikselasi acak, perubahan kecerahan/kontras, dan rotasi kecil). Akibatnya, banyak citra latih merupakan variasi dari foto asli yang sama. Sifat inilah yang membuat akurasi validasi hampir selalu mendekati 100% pada semua skenario (Subbab 6.d).

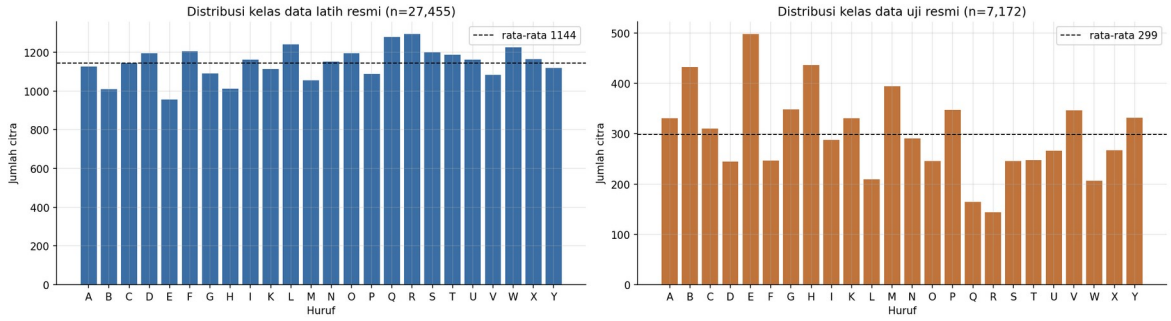
### d. Seberapa kompleks dataset tersebut?

1. Berukuran sedang (34.627 citra) dengan 24 kelas dan 784 fitur per citra.
2. Data latih relatif seimbang (rasio 1,35), tetapi data uji timpang (rasio 3,46).
3. Kelas bersifat fine-grained: beberapa pasangan huruf hanya berbeda pada detail kecil.
4. Latar dan kondisi pemotretan relatif seragam, sehingga dataset ini tergolong "bersih". Akibatnya, akurasi tinggi pada data uji belum menjamin performa yang sama pada foto dunia nyata (lihat Subbab 5.e dan 6.e).

## 3. Exploratory Data Analysis (EDA)

### a. Distribusi label/kelas

Gambar 1 memperlihatkan distribusi jumlah citra per huruf. Data latih relatif seimbang (957–1.294 citra per kelas, rasio 1,35). Data uji cukup timpang dengan rasio 3,46: kelas terbanyak E (498 citra) dan tersedikit R (144 citra). Karena itu, macro-F1 dilaporkan di samping accuracy agar setiap huruf berbobot sama.




*Gambar 1 - Distribusi kelas pada data latih dan uji resmi*

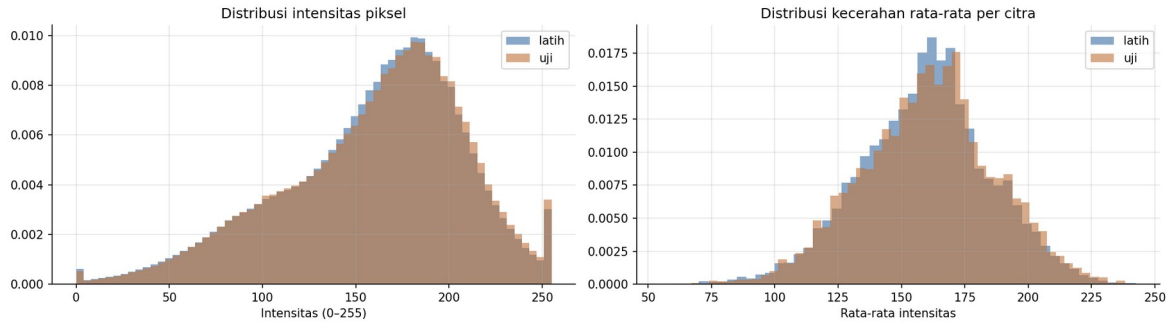
### b. Word count

Tidak relevan, karena dataset berupa citra dan tidak mengandung teks.

### c. Histogram fitur numerik

Untuk data citra, fitur numeriknya adalah intensitas piksel. Gambar 2 membandingkan histogram intensitas piksel dan kecerahan rata-rata per citra. Rata-rata ± simpangan baku piksel adalah

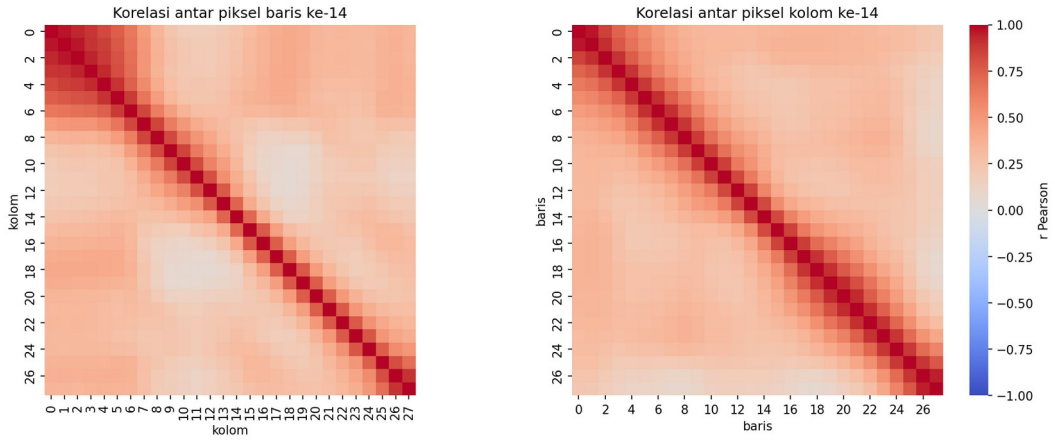
159,3 ± 48,8 (latih) dan 160,6 ± 49,0 (uji). Distribusi global latih dan uji hampir sama, tetapi kecerahan antarcitra tersebar lebar. Karena itu, augmentasi kecerahan dan kontras digunakan.




*Gambar 2 - Histogram intensitas piksel dan kecerahan rata-rata citra latih vs uji*

### d. Korelasi antarfitur

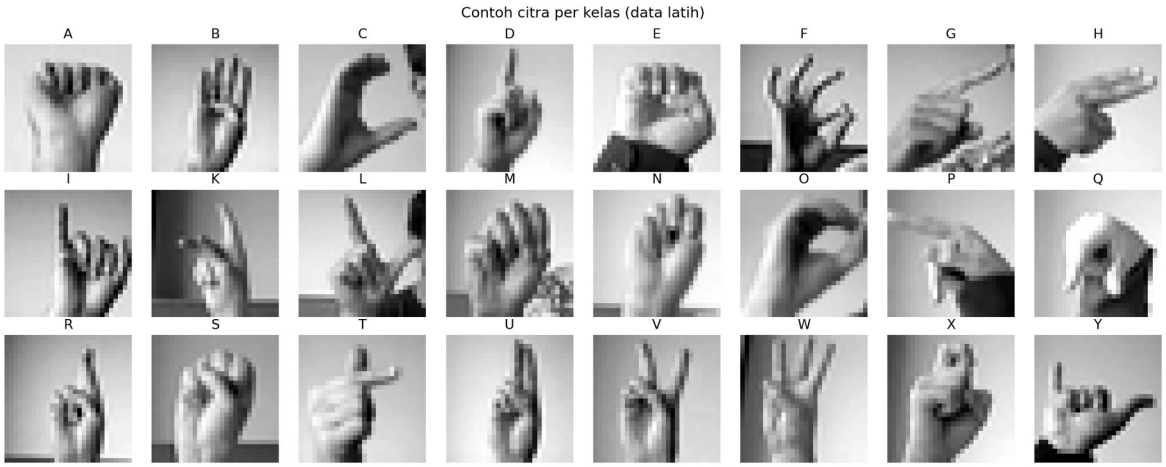
Gambar 3 menunjukkan korelasi Pearson antarpiksel pada baris dan kolom tengah. Piksel bertetangga berkorelasi sangat kuat (rata-rata r = 0,857), sedangkan piksel berjarak 10 jauh lebih lemah (r = 0,246). Struktur korelasi yang sangat lokal ini menjadi alasan utama penggunaan CNN: konvolusi memanfaatkan keterhubungan lokal, sedangkan DNN memperlakukan setiap piksel sebagai fitur independen.



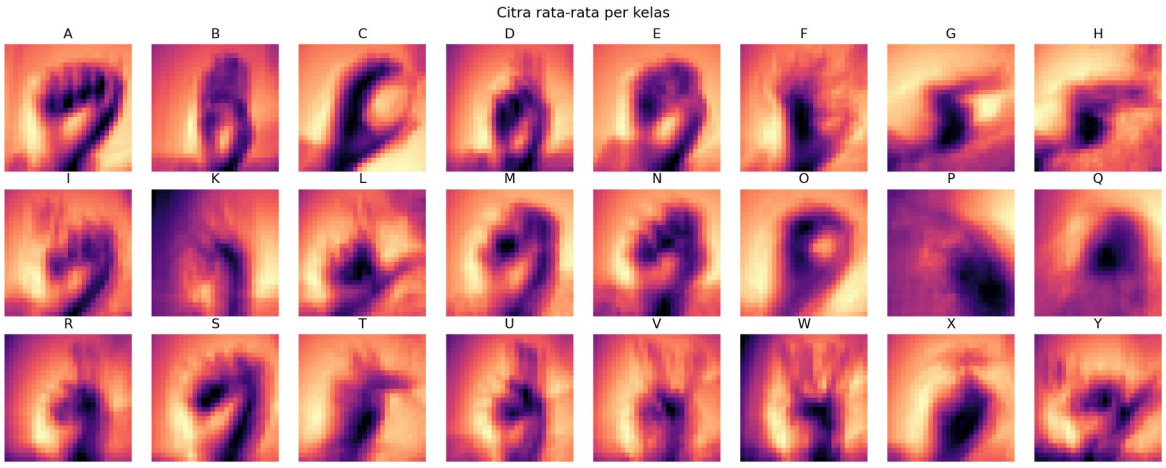

*Gambar 3 - Korelasi antarpiksel pada baris ke-14 dan kolom ke-14*

### e. Contoh sampel data per kelas

Gambar 4 menampilkan satu contoh citra untuk setiap huruf. Gambar 5 menampilkan citra rata-rata per kelas, yang memperlihatkan pola bentuk tangan khas tiap huruf, misalnya lengkungan C dan O atau jari yang menunjuk ke samping pada G dan H.




*Gambar 4 - Contoh citra untuk setiap huruf*




*Gambar 5 - Citra rata-rata setiap kelas*

## f. Temuan penting dari EDA yang memengaruhi preprocessing

*Tabel 4 - Temuan EDA dan dampaknya pada preprocessing*

| Temuan EDA | Keputusan preprocessing/pemodelan |
|---|---|
| Tidak ada missing value; semua piksel 0–255; 0 duplikat persis dan 0 citra uji yang identik dengan citra latih | Tidak ada baris yang dihapus; cukup validasi rentang dan pemetaan label |
| Data uji timpang (rasio 3,46) | Macro-F1 dipakai sebagai metrik pendamping accuracy |
| Kecerahan dan posisi tangan bervariasi | Augmentasi brightness, contrast, rotasi, zoom, dan translasi pada tf.data |
| Korelasi piksel sangat lokal (r tetangga 0,857) | Arsitektur CNN dengan konvolusi 3×3 dan pooling |
| Data latih berupa variasi dari foto asli yang sama | Validasi tidak dipakai sebagai satu-satunya ukuran; data uji resmi dipisahkan sepenuhnya |
| Label asli 0–24 tanpa 9 | Re-indeks label ke 0–23 |

## 4. Preprocessing Data

### a. Pembersihan data

1. Pemeriksaan missing value: 0 pada data latih dan 0 pada data uji, sehingga tidak ada imputasi.
2. Pemeriksaan duplikat dengan hash baris piksel: 0 duplikat persis di data latih dan 0 citra uji yang identik dengan data latih.
3. Validasi data rusak: semua nilai piksel berada pada rentang 0–255 dan setiap baris berukuran 784 piksel.
4. Validasi label: hanya 24 nilai label yang sah (0–24 tanpa 9), lalu dipetakan ulang ke indeks 0–23 agar sesuai dengan 24 neuron softmax.

### b. Normalisasi dan rescale piksel

Piksel diubah ke float32 lalu dibagi 255 (x' = x/255) sehingga berada pada rentang [0, 1]. Konstanta 255 tidak dihitung dari data, sehingga tidak ada kebocoran statistik. Vektor 784 diubah menjadi tensor (28, 28, 1), dan satu batch menjadi tensor rank-4 (B, 28, 28, 1). Resize tidak diperlukan karena dataset sudah 28×28. Pada aplikasi, foto pengguna diubah ke grayscale, dipotong persegi di tengah, di-resize ke 28×28 (interpolasi box), lalu dibagi 255. Data dialirkan dengan pipeline tf.data berikut.

```
ds = tf.data.Dataset.from_tensor_slices((X, y)).cache()
ds = ds.shuffle(len(X), seed=SEED, reshuffle_each_iteration=True)    # hanya data latih
ds = ds.map(photometric_augment, num_parallel_calls=AUTOTUNE)        # hanya E5
ds = ds.batch(128)
ds = ds.map(lambda x, y: (geo_augment(x, training=True), y), num_parallel_calls=AUTOTUNE)
# E5
ds = ds.prefetch(AUTOTUNE)
```

Hasil pengukuran throughput: pipeline naif (tanpa cache, shuffle, dan prefetch) mencapai 422.903 citra/detik, pipeline cache+shuffle+prefetch 309.063 citra/detik, dan pipeline dengan augmentasi 9.455 citra/detik. Pipeline naif sedikit lebih cepat karena seluruh data sudah berada di RAM, sehingga cache tidak memberi keuntungan, sementara shuffle dengan buffer seluruh data menambah biaya per elemen. Shuffle tetap dipertahankan karena wajib untuk SGD. Augmentasi jauh lebih mahal: waktu per epoch naik dari ±2,9 detik (E4) menjadi ±4,1 detik (E5), sehingga pada E5 pipeline data mendekati titik hambat (bottleneck). Perbaikan ke depan adalah memindahkan augmentasi ke dalam model agar dijalankan di GPU.

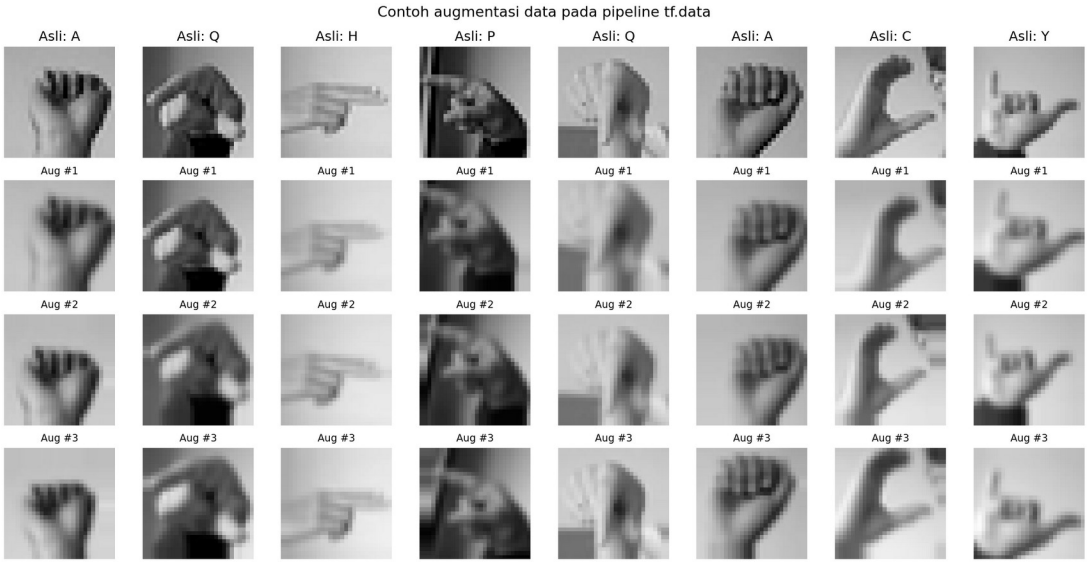
### c. Augmentasi data

Augmentasi hanya diterapkan pada data latih skenario E5, dengan parameter pada Tabel 5. Contoh hasilnya ditunjukkan pada Gambar 6.

*Tabel 5 - Jenis dan parameter augmentasi data*

| Transformasi | Parameter | Alasan |
|---|---|---|

| | | |
|---|---|---|
| RandomRotation | faktor 0,03 (±10,8°) | Variasi kemiringan tangan |
| RandomZoom | ±10% | Variasi jarak tangan ke kamera |
| RandomTranslation | ±10% tinggi & lebar | Variasi posisi tangan di bingkai |
| random_brightness | $\Delta \leq 0{,}10$ | Variasi pencahayaan |
| random_contrast | faktor 0,8–1,2 | Variasi kontras kamera |
| Flip horizontal | Tidak digunakan | Membalik citra mengubah tangan kanan menjadi kiri |




*Gambar 6 - Citra asli (baris pertama) dan tiga variasi hasil augmentasi*

### d. Penanganan missing value, outlier, dan ketidakseimbangan kelas

1. Missing value: tidak ada, sehingga tidak diperlukan imputasi.
2. Outlier: tidak ada nilai piksel di luar rentang. Citra dengan pencahayaan ekstrem tidak dibuang, tetapi diantisipasi dengan augmentasi brightness/contrast.
3. Ketidakseimbangan kelas: data latih sudah relatif seimbang sehingga tidak diperlukan oversampling atau class weight; ketimpangan data uji ditangani dengan pelaporan macro-F1 dan metrik per kelas.

### e. Pembagian data dan pencegahan kebocoran data

*Tabel 6 - Pembagian data*

| Subset | Sumber | Jumlah | Proporsi | Kegunaan |
|---|---|---|---|---|
| Latih | Data latih resmi (stratified) | 24.709 | 90% data latih | Memperbarui bobot |
| Validasi | Data latih resmi (stratified) | 2.746 | 10% data latih | Early stopping, scheduler, checkpoint, ambang $\tau$ |
| Uji | Data uji resmi | 7.172 | Terpisah | Evaluasi akhir saja |

1. Data uji resmi tidak pernah dipakai untuk pelatihan, early stopping, pemilihan hyperparameter, maupun penentuan ambang $\tau$.
2. Pembagian latih/validasi dilakukan secara stratified dengan random_state=42.

3. Ambang τ (fitur unik 1) ditentukan dari data validasi yang diaugmentasi, bukan dari data uji.
4. Model akhir (E5) ditetapkan sejak awal berdasarkan rancangan, bukan dipilih dengan melihat skor uji.
5. Pada studi aktivasi dan optimizer, akurasi uji hanya dilaporkan; keputusan diambil berdasarkan data validasi.

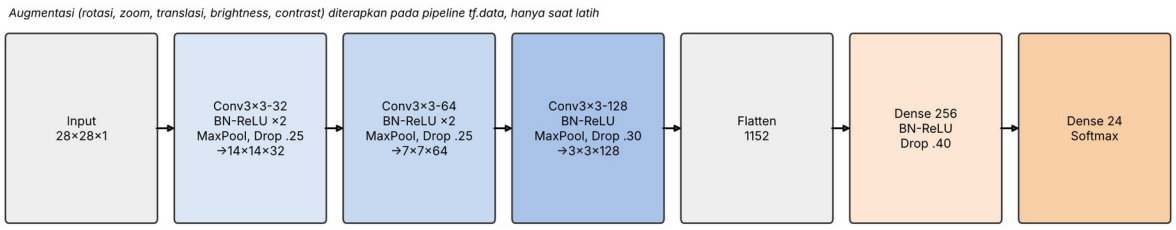
## 5. Model yang Digunakan

### a. Jenis model

Dua keluarga model digunakan sesuai ketentuan. DNN (Multi-Layer Perceptron) menjadi baseline E1–E2, dan CNN menjadi model utama E3–E5. Model akhir adalah CNN tiga blok konvolusi yang dilatih dari awal (E5). Transfer learning (ResNet, VGG, MobileNet, EfficientNet) tidak dipakai karena tiga alasan: input berupa citra grayscale 28×28 yang sangat berbeda dari input model pretrained (RGB ±224×224); CNN kecil terbukti sudah cukup (akurasi uji 100%); dan model kecil (441.912 parameter) jauh lebih ringan untuk deployment.

### b. Arsitektur model

Diagram arsitektur CNN akhir ditunjukkan pada Gambar 7, dan rincian setiap lapisannya pada Tabel 7. Setiap konvolusi 3×3 (padding same) diikuti BatchNormalization dan ReLU. Setiap blok diakhiri MaxPooling 2×2 dan Dropout. Lapisan sebelum BatchNorm tidak memakai bias karena perannya sudah digantikan parameter β. Output berupa softmax 24 kelas.




*Gambar 7 - Diagram arsitektur CNN akhir (E5)*

*Tabel 7 - Rincian lapisan CNN akhir (B = ukuran batch)*

| Lapisan | Tipe | Ukuran output | Parameter |
|---|---|---|---|
| b1c1_conv | Conv2D | (B, 28, 28, 32) | 288 |
| b1c1_bn | BatchNormalization | (B, 28, 28, 32) | 128 |
| b1c1_relu | Activation | (B, 28, 28, 32) | 0 |
| b1c2_conv | Conv2D | (B, 28, 28, 32) | 9.216 |
| b1c2_bn | BatchNormalization | (B, 28, 28, 32) | 128 |
| b1c2_relu | Activation | (B, 28, 28, 32) | 0 |
| b1_pool | MaxPooling2D | (B, 14, 14, 32) | 0 |
| b1_drop | Dropout | (B, 14, 14, 32) | 0 |
| b2c1_conv | Conv2D | (B, 14, 14, 64) | 18.432 |
| b2c1_bn | BatchNormalization | (B, 14, 14, 64) | 256 |
| b2c1_relu | Activation | (B, 14, 14, 64) | 0 |
| b2c2_conv | Conv2D | (B, 14, 14, 64) | 36.864 |

| | | | |
|---|---|---|---|
| b2c2_bn | BatchNormalization | (B, 14, 14, 64) | 256 |
| b2c2_relu | Activation | (B, 14, 14, 64) | 0 |
| b2_pool | MaxPooling2D | (B, 7, 7, 64) | 0 |
| b2_drop | Dropout | (B, 7, 7, 64) | 0 |
| b3c1_conv | Conv2D | (B, 7, 7, 128) | 73.728 |
| b3c1_bn | BatchNormalization | (B, 7, 7, 128) | 512 |
| b3c1_relu | Activation | (B, 7, 7, 128) | 0 |
| b3_pool | MaxPooling2D | (B, 3, 3, 128) | 0 |
| b3_drop | Dropout | (B, 3, 3, 128) | 0 |
| flatten | Flatten | (B, 1152) | 0 |
| fc_dense | Dense | (B, 256) | 294.912 |
| fc_bn | BatchNormalization | (B, 256) | 1.024 |
| fc_relu | Activation | (B, 256) | 0 |
| fc_drop | Dropout | (B, 256) | 0 |
| output_softmax | Dense | (B, 24) | 6.168 |
| Total | | | 441.912 |

Sebagai pembanding, Tabel 8 merangkum arsitektur kelima skenario.

*Tabel 8 - Ringkasan arsitektur skenario E1–E5*

| ID | Jenis | Arsitektur ringkas | Parameter |
|---|---|---|---|
| E1 | DNN | Flatten → Dense 256 (ReLU) → Dense 128 (ReLU) → Softmax 24 | 236.952 |
| E2 | DNN | Flatten → [Dense 512/256/128 + BN + ReLU + Dropout 0,3] → Softmax 24 | 571.928 |
| E3 | CNN | Conv32 → Pool → Conv64 → Pool → Dense 128 → Softmax 24 | 423.448 |
| E4 | CNN | Arsitektur pada tabel di atas, tanpa augmentasi | 441.912 |
| E5 | CNN | Arsitektur E4 + augmentasi pada tf.data (model akhir) | 441.912 |

## c. Konfigurasi pelatihan

*Tabel 9 - Konfigurasi pelatihan (ReduceLR = ReduceLROnPlateau: faktor 0,5, patience 3, min_lr 1e-5)*

| Konfigurasi | E1 | E2 | E3 | E4 | E5 |
|---|---|---|---|---|---|
| Optimizer | SGD | Adam | Adam | Adam | Adam |
| Learning rate awal | 0,01 | 0,001 | 0,001 | 0,001 | 0,001 |
| LR scheduler | – | ReduceLR | – | ReduceLR | ReduceLR |
| Batch size | 128 | 128 | 128 | 128 | 128 |
| Epoch maksimum | 30 | 40 | 30 | 40 | 50 |
| Epoch berjalan (terbaik) | 30 (28) | 40 (39) | 30 (29) | 40 (40) | 40 (30) |
| EarlyStopping (patience) | 5 | 8 | 5 | 8 | 10 |
| Dropout | – | 0,3 | – | 0,25–0,40 | 0,25–0,40 |
| BatchNorm | – | √ | – | √ | √ |
| Augmentasi | – | – | – | – | √ |
| Fungsi loss | Sparse CCE | Sparse CCE | Sparse CCE | Sparse CCE | Sparse CCE |
| Inisialisasi bobot | He normal | He normal | He normal | He normal | He normal |

Callback yang digunakan adalah EarlyStopping (monitor val_loss, restore_best_weights), ModelCheckpoint (bobot dengan val_loss terendah), ReduceLROnPlateau, CSVLogger dan TensorBoard untuk monitoring, serta LRLogger kustom yang mencatat learning rate setiap epoch. Fungsi loss sparse categorical cross-entropy, $L = -(1/N) \Sigma \log p(y_i \mid x_i)$, dipakai karena label berupa indeks bilangan bulat dan output berupa distribusi softmax.

**d. Alasan pemilihan model dan arsitektur**

1. **Bertahap dari konsep dasar.** Perceptron dan MLP (DNN) dipakai untuk memahami backpropagation dan menjadi baseline, lalu ditingkatkan menjadi CNN. Setiap peningkatan diukur sehingga kontribusinya terlihat (Subbab 6.d).
2. **CNN sesuai karakter data.** EDA menunjukkan korelasi piksel yang sangat lokal. Konvolusi memanfaatkan keterhubungan lokal dan berbagi bobot (LeCun et al., 1998). Terbukti, CNN baseline E3 mengungguli DNN E2 sebesar 9,31 poin persentase meskipun memiliki parameter lebih sedikit.
3. **Konvolusi 3×3 bertumpuk.** Dua konvolusi 3×3 memberi receptive field 5×5 dengan parameter lebih sedikit dan nonlinieritas lebih banyak (Simonyan & Zisserman, 2015).
4. **BatchNorm dan Dropout.** BatchNorm menstabilkan dan mempercepat pelatihan (Ioffe & Szegedy, 2015), sedangkan Dropout mencegah co-adaptation antarneuron (Srivastava et al., 2014). Keduanya menaikkan akurasi uji dari 91,49% (E3) menjadi 99,94% (E4).
5. **ReLU.** Pada studi DNN, tanh mencapai akurasi validasi akhir tertinggi (100,00%) dengan ReLU sangat dekat (99,71%), sedangkan sigmoid tertinggal (92,46%). ReLU tetap dipilih untuk CNN karena selisihnya kecil, tidak mengalami saturasi untuk input positif (Nair & Hinton, 2010), dan sudah menjadi pasangan standar BatchNorm dengan inisialisasi He (He et al., 2015).
6. **Adam.** Pada studi optimizer, RMSProp dan Adam sama-sama mencapai loss validasi ±0,0001. Adam dipilih karena konvergensinya paling stabil: akurasi validasinya 99,96% sejak epoch ke-3 dan bertahan 100% sejak epoch ke-4 tanpa penurunan, sedangkan RMSProp sempat turun ke 97,89% pada epoch ke-7 (Kingma & Ba, 2015).
7. **Augmentasi untuk generalisasi.** Augmentasi mengatasi pergeseran distribusi antara data latih dan uji (Shorten & Khoshgoftaar, 2019) dan menghilangkan 4 kesalahan E4 yang tersisa sehingga E5 mencapai 100%.

**e. Dua fitur unik**

***Fitur unik 1: Confidence threshold dan deteksi input di luar domain (OOD)***

**Apa fiturnya.** Prediksi hanya diterima jika probabilitas softmax tertinggi (maximum softmax probability, MSP) ≥ ambang τ. Jika di bawah τ, aplikasi menjawab "tidak yakin / bukan isyarat huruf ASL" dan tidak memaksakan satu huruf.

**Mengapa dibutuhkan.** Softmax selalu memaksakan salah satu dari 24 huruf, termasuk untuk foto yang bukan tangan. Pada aplikasi nyata, jawaban salah yang meyakinkan lebih merugikan daripada penolakan. MSP adalah baseline deteksi input di luar distribusi yang sederhana (Hendrycks & Gimpel, 2017).

**Bagaimana implementasinya.** (1) Data validasi diaugmentasi untuk mensimulasikan pergeseran distribusi. (2) Dipilih τ terkecil yang membuat akurasi prediksi yang diterima ≥ 99,5%, dengan

batas bawah τ = 0,5. (3) τ disimpan di config.json dan dipakai aplikasi; pengguna juga dapat mengubahnya lewat slider. Karena model sudah benar 100% pada validasi teraugmentasi, kriteria akurasi terpenuhi di semua ambang sehingga τ jatuh ke batas bawah, yaitu τ = 0,50.

**Bukti dampaknya.** Tabel 10 dan Tabel 11 membandingkan kondisi tanpa dan dengan fitur ini.

*Tabel 10 - Dampak fitur unik 1 pada data uji*

| Metrik (data uji resmi) | Tanpa ambang | Dengan ambang τ = 0,50 |
|---|---|---|
| Akurasi prediksi yang diterima | 100,00% | 100,00% |
| Coverage (prediksi yang diterima) | 100,00% | 100,00% |
| Kesalahan yang lolos ke pengguna | 0 | 0 |
| Prediksi benar yang ikut ditolak | – | 0 |

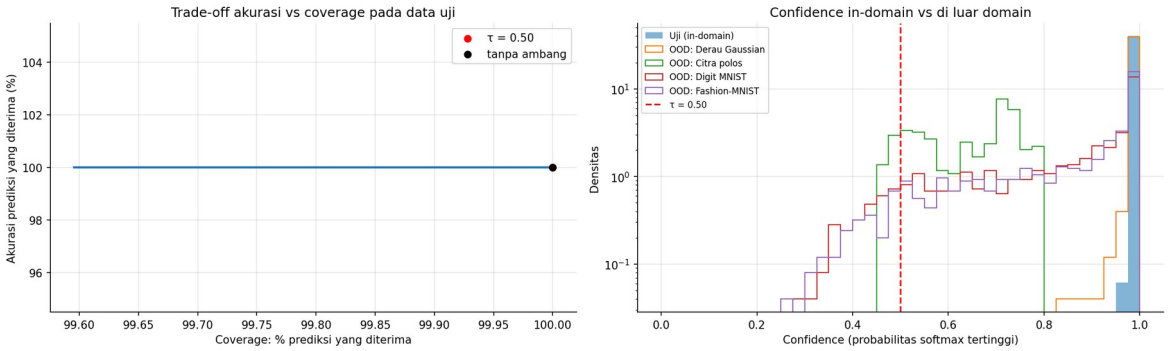
*Tabel 11 - Kemampuan menolak input di luar domain (tanpa fitur ini, 0% ditolak)*

| Set input di luar domain | Jumlah | % ditolak | Rata-rata confidence | AUROC |
|---|---|---|---|---|
| Derau Gaussian | 1.000 | 0,0% | 0,998 | 0,810 |
| Citra polos | 1.000 | 10,8% | 0,643 | 1,000 |
| Digit MNIST | 1.000 | 7,0% | 0,836 | 0,990 |
| Fashion-MNIST | 1.000 | 5,4% | 0,855 | 0,977 |

Hasilnya perlu dibaca secara jujur. Pada data uji, fitur ini tidak mengubah apa pun karena model akhir tidak membuat kesalahan sama sekali dan semua prediksinya sangat yakin (rata-rata confidence 99,98%; Gambar 8 memperlihatkan seluruh confidence uji berada di atas ±0,9). Pada input di luar domain, dengan τ = 0,5 fitur ini hanya menolak 10,8% citra polos, 5,4% Fashion-MNIST, 7,0% digit MNIST, dan 0,0% derau Gaussian.

Temuan paling penting ada pada derau Gaussian: rata-rata confidence-nya 0,998, hampir sama dengan citra uji asli (0,9998), dan tidak satu pun tertolak pada τ = 0,5. AUROC-nya 0,810 menunjukkan derau hanya dapat dipisahkan dengan ambang yang sangat tinggi (±0,99), dan itu pun tidak seluruhnya. Model memprediksi derau sebagai huruf P dengan keyakinan ±100% (Gambar 9). Ini adalah keterbatasan MSP yang sudah dikenal: CNN dapat memberi confidence sangat tinggi pada input yang sama sekali asing (Hendrycks & Gimpel, 2017). Sebaliknya, AUROC untuk digit MNIST (0,990), Fashion-MNIST (0,977), dan citra polos (1,000) cukup tinggi. Artinya, confidence pada ketiga jenis input itu umumnya lebih rendah daripada data asli, sehingga ambang yang lebih tinggi (misalnya ±0,9, karena seluruh data uji berada di atasnya) akan menolak jauh lebih banyak input tersebut tanpa mengorbankan data uji. Masalahnya terletak pada cara pemilihan τ yang hanya mengejar target akurasi, bukan pada konsep fiturnya.

Rencana perbaikan fitur ini: (1) memilih τ berdasarkan target penolakan pada set OOD validasi (misalnya ≥95% citra polos/MNIST ditolak dengan coverage data valid ≥99%); (2) menambahkan pemeriksaan input sederhana sebelum model, misalnya energi frekuensi tinggi atau variansi lokal piksel untuk mendeteksi derau; dan (3) melatih kelas tambahan "bukan tangan" atau memakai temperature scaling agar confidence lebih terkalibrasi.




*Gambar 8 - Trade-off akurasi–coverage dan distribusi confidence (dalam vs luar domain)*




*Gambar 9 - Contoh input di luar domain dan keputusan model pada τ = 0,5 (merah = tidak tertolak)*

### *Fitur unik 2: Kompresi model ke TensorFlow Lite*

**Apa fiturnya.** Model Keras dikonversi ke TensorFlow Lite (LiteRT) dalam empat varian: FP32, FP16, dynamic range (bobot INT8), dan full integer INT8 yang dikalibrasi dengan representative dataset.

**Mengapa dibutuhkan.** Aplikasi web gratis (Hugging Face Spaces CPU) dan perangkat seluler membutuhkan model kecil yang cepat dimuat dan diinferensi. Model TFLite cukup dijalankan dengan runtime ai-edge-litert yang ringan, tanpa TensorFlow penuh. Kuantisasi INT8 dapat mengecilkan bobot sekitar 4× dengan penurunan akurasi kecil (Jacob et al., 2018).

**Bagaimana implementasinya.** Model diekspor dengan model.export() ke SavedModel, lalu dikonversi dengan tf.lite.TFLiteConverter (optimizations = DEFAULT, supported_types = float16, atau representative_dataset untuk INT8). Setiap varian dievaluasi pada seluruh 7.172 citra uji menggunakan Interpreter. Varian terkecil yang akurasinya turun tidak lebih dari 0,2 poin dipilih untuk deployment.

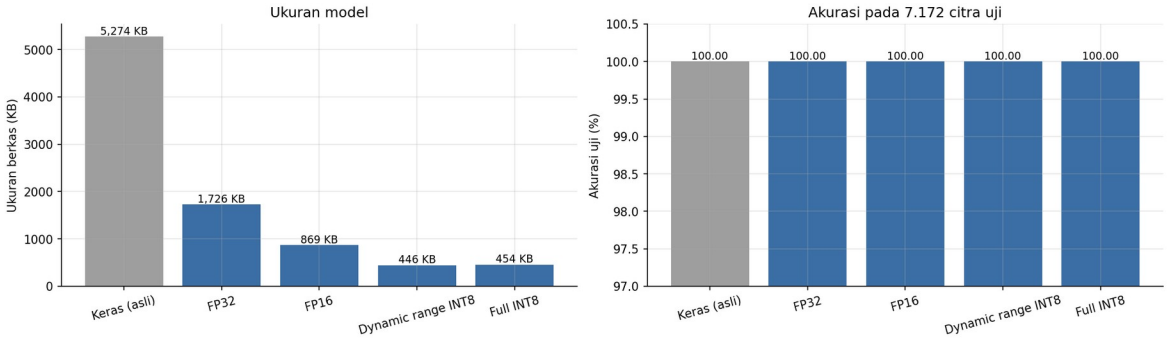
**Bukti dampaknya.** Tabel 12 dan Gambar 10 membandingkan model asli dengan varian TFLite.

*Tabel 12 - Dampak fitur unik 2: ukuran, akurasi, latensi (★ = varian deployment)*

| Model | Ukuran (KB) | Akurasi uji | Macro-F1 | Sama dengan Keras | Latensi (ms/citra) |
|---|---|---|---|---|---|
| Keras .keras (asli) | 5.274,3 | 100,00% | 100,00% | 100,00% | 19,523 |
| FP32 | 1.726,2 | 100,00% | 100,00% | 100,00% | 0,574 |
| FP16 | 869,0 | 100,00% | 100,00% | 100,00% | 0,624 |
| Dynamic range INT8 ★ | 446,0 | 100,00% | 100,00% | 100,00% | 0,652 |
| Full INT8 | 453,5 | 100,00% | 100,00% | 100,00% | 0,925 |

Varian Dynamic range INT8 berukuran 446,0 KB, atau 11,8× lebih kecil daripada berkas .keras (5.274,3 KB), tanpa penurunan akurasi sama sekali (100,00%) dan dengan 100,00% prediksi

identik dengan model Keras. Sebagai catatan, berkas .keras juga menyimpan state optimizer Adam. Ukuran bobot murninya terlihat pada TFLite FP32 (1.726,2 KB ≈ 441.912 parameter × 4 byte), sehingga efek kuantisasi INT8 terhadap bobot adalah 3,9×, sesuai teori (±4×). Varian Full INT8 sedikit lebih besar daripada dynamic range karena menambahkan operasi kuantisasi/dekuantisasi untuk input/output float. Latensinya 0,652 ms per citra, jauh di bawah pemanggilan Keras satu per satu (19,52 ms). Namun, angka Keras tersebut didominasi overhead pemanggilan eager; dalam mode batch Keras mencapai 0,037 ms per citra. Untuk aplikasi yang memproses satu foto per permintaan, TFLite jelas lebih sesuai.




*Gambar 10 - Ukuran berkas dan akurasi uji model Keras vs varian TFLite*

## 6. Hasil Evaluasi

### a. Metrik evaluasi yang digunakan dan alasannya

- **Accuracy:** proporsi prediksi benar; ukuran utama yang mudah dipahami.
- **Precision, recall, dan F1-score per kelas:** precision = TP/(TP+FP), recall = TP/(TP+FN), dan F1 = 2PR/(P+R). Ketiganya mengungkap huruf mana yang sering salah (Sokolova & Lapalme, 2009).
- **Macro-F1:** rata-rata F1 dengan bobot sama per kelas; penting karena data uji timpang.
- **Weighted-F1 dan loss uji:** pelengkap untuk melihat performa berbobot support dan tingkat keyakinan model.
- **Confusion matrix 24×24:** memperlihatkan pasangan huruf yang tertukar.
- **Coverage, AUROC OOD, ukuran model, dan latensi:** untuk mengukur dampak dua fitur unik.

### b. Nilai metrik pada data testing

*Tabel 13 - Nilai metrik model akhir pada data uji*

| Metrik (model E5, 7.172 citra uji) | Nilai |
|---|---|
| Accuracy | 100,00% |
| Prediksi benar / total | 7.172 / 7.172 |
| Macro precision | 100,00% |
| Macro recall | 100,00% |
| Macro F1-score | 100,00% |

| | |
|---|---|
| Weighted F1-score | 100,00% |
| Loss uji (sparse CCE) | 0,00023 |
| Rata-rata confidence prediksi | 99,98% |
| Latensi Keras | 19,52 ms/citra (tunggal), 0,037 ms/citra (batch) |

Model akhir mengklasifikasikan seluruh 7.172 citra uji dengan benar, sehingga precision, recall, dan F1-score bernilai 100% untuk setiap huruf (Tabel 14). Hasil ini konsisten dengan karakter dataset yang berlatar relatif seragam dan dengan augmentasi yang menutup pergeseran distribusi. Meski begitu, angka ini adalah performa pada distribusi Sign Language MNIST. Hasil fitur unik 1 dan Grad-CAM menunjukkan bahwa performa pada foto dunia nyata perlu diuji terpisah.

*Tabel 14 - Classification report per huruf (E5)*

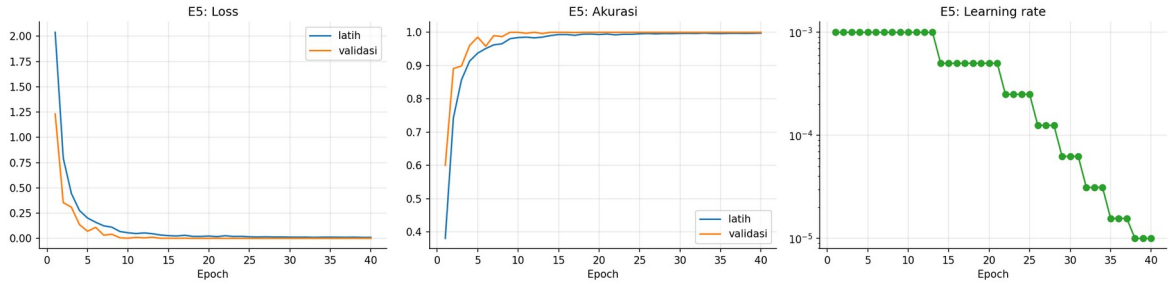
| Huruf | Precision | Recall | F1-score | Support |
|---|---|---|---|---|
| A | 100,00% | 100,00% | 100,00% | 331 |
| B | 100,00% | 100,00% | 100,00% | 432 |
| C | 100,00% | 100,00% | 100,00% | 310 |
| D | 100,00% | 100,00% | 100,00% | 245 |
| E | 100,00% | 100,00% | 100,00% | 498 |
| F | 100,00% | 100,00% | 100,00% | 247 |
| G | 100,00% | 100,00% | 100,00% | 348 |
| H | 100,00% | 100,00% | 100,00% | 436 |
| I | 100,00% | 100,00% | 100,00% | 288 |
| K | 100,00% | 100,00% | 100,00% | 331 |
| L | 100,00% | 100,00% | 100,00% | 209 |
| M | 100,00% | 100,00% | 100,00% | 394 |
| N | 100,00% | 100,00% | 100,00% | 291 |
| O | 100,00% | 100,00% | 100,00% | 246 |
| P | 100,00% | 100,00% | 100,00% | 347 |
| Q | 100,00% | 100,00% | 100,00% | 164 |
| R | 100,00% | 100,00% | 100,00% | 144 |
| S | 100,00% | 100,00% | 100,00% | 246 |
| T | 100,00% | 100,00% | 100,00% | 248 |
| U | 100,00% | 100,00% | 100,00% | 266 |
| V | 100,00% | 100,00% | 100,00% | 346 |
| W | 100,00% | 100,00% | 100,00% | 206 |
| X | 100,00% | 100,00% | 100,00% | 267 |
| Y | 100,00% | 100,00% | 100,00% | 332 |

### c. Grafik loss dan akurasi data training dan validation

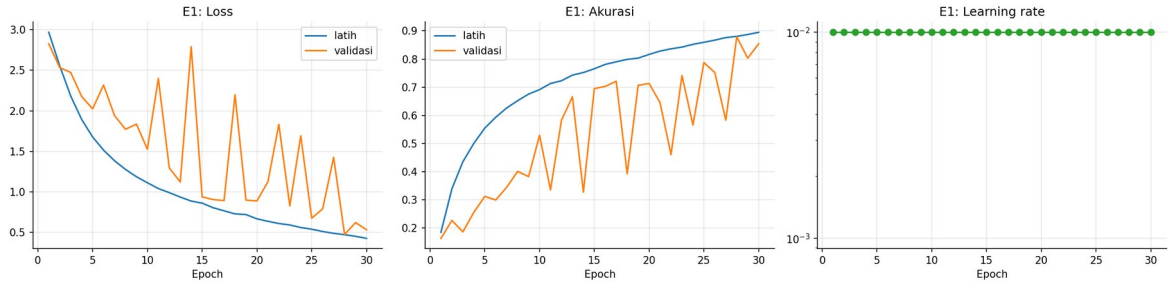
Gambar 11 menampilkan kurva loss, akurasi, dan learning rate model akhir E5. Pelatihan berjalan 40 epoch dan dihentikan EarlyStopping; epoch terbaik adalah ke-30. Loss latih dan validasi turun bersama tanpa divergensi. Pada epoch terbaik, akurasi latih 99,68% sedikit lebih rendah daripada akurasi validasi 100,00%, karena data latih diaugmentasi dan dropout hanya aktif saat latih. Jadi tidak ada indikasi overfitting. Learning rate diturunkan 7 kali oleh ReduceLROnPlateau, dari 0,001 menjadi 1,00e-05, setiap kali loss validasi stagnan.

Sebagai pembanding, Gambar 12 menampilkan kurva DNN baseline E1 (SGD tanpa momentum). Akurasi validasinya naik lambat dan berfluktuasi tajam, dan pada akhir pelatihan

akurasi latih (88,05%) dan validasinya (87,87%) sama-sama belum tinggi. Ini adalah tanda underfitting.




*Gambar 11 - Kurva loss, akurasi, dan learning rate (latih vs validasi) model E5*



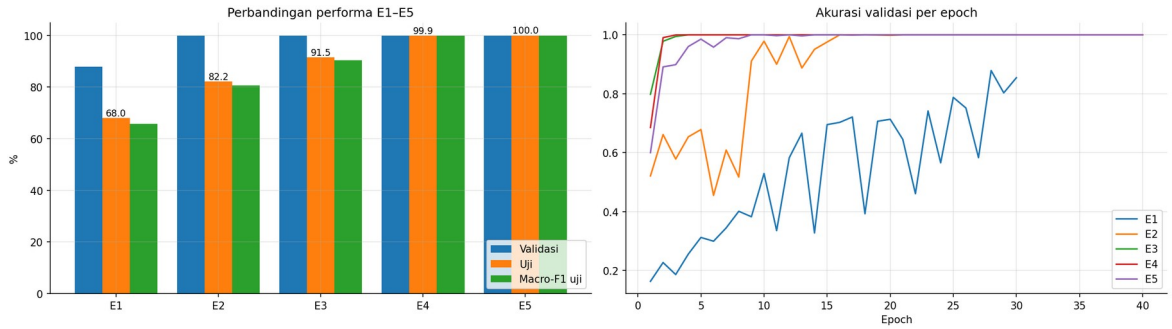

*Gambar 12 - Kurva loss dan akurasi (latih vs validasi) DNN baseline E1*

## d. Perbandingan antarskenario

Tabel 15 dan Gambar 13 membandingkan kelima skenario. Gap adalah selisih akurasi validasi dan uji.

*Tabel 15 - Hasil seluruh skenario*

| ID | Skenario | Epoch (terbaik) | Waktu (s) | Acc latih | Acc val. | Acc uji | Macro-F1 | Gap val–uji |
|---|---|---|---|---|---|---|---|---|
| E1 | MLP Baseline | 30 (28) | 27 | 88,05% | 87,87% | 67,99% | 65,67% | 19,89 pp |
| E2 | MLP + BatchNorm + Dropout + Adam | 40 (39) | 46 | 99,86% | 100,00% | 82,18% | 80,59% | 17,82 pp |
| E3 | CNN Baseline | 30 (29) | 45 | 100,00% | 100,00% | 91,49% | 90,32% | 8,51 pp |
| E4 | CNN + BatchNorm + Dropout | 40 (40) | 114 | 99,97% | 100,00% | 99,94% | 99,94% | 0,06 pp |
| E5 | CNN + BatchNorm + Dropout + Augmentasi | 40 (30) | 165 | 99,68% | 100,00% | 100,00% | 100,00% | 0,00 pp |




*Gambar 13 - Perbandingan performa skenario E1–E5 dan kurva akurasi validasi*

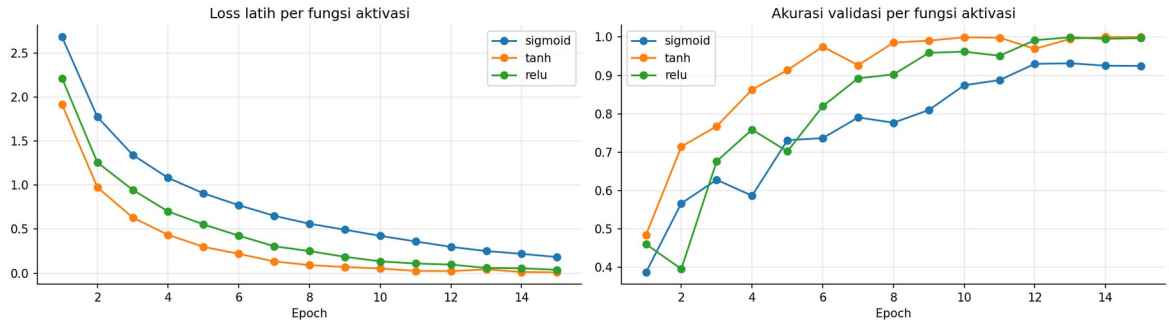
1. E1 → E2 (regularisasi DNN + Adam): akurasi uji naik 14,19 poin persentase menjadi 82,18%. Namun, akurasi validasi E2 mencapai 100,00%, sehingga gap val–uji tetap besar (17,82 pp). DNN menghafal pola piksel pada posisi tertentu dan gagal pada data uji.
2. E2 → E3 (DNN diganti CNN): akurasi uji naik 9,31 poin persentase menjadi 91,49% dengan parameter lebih sedikit (423.448 vs 571.928). Ini bukti kekuatan bias induktif konvolusi.
3. E3 → E4 (blok konvolusi bertumpuk + BN + Dropout + scheduler): akurasi uji naik 8,45 poin persentase menjadi 99,94% (±4 kesalahan), dan gap turun dari 8,51 pp menjadi 0,06 pp.
4. E4 → E5 (augmentasi): akurasi uji naik 0,06 poin persentase menjadi 100,00% dan gap hilang sepenuhnya, dengan biaya waktu per epoch naik dari ±2,9 s menjadi ±4,1 s.
5. Secara keseluruhan, E1 → E5 menaikkan akurasi uji sebesar 32,01 poin persentase, dari 67,99% menjadi 100,00%.

### *Studi fungsi aktivasi dan optimizer*

*Tabel 16 - Perbandingan fungsi aktivasi (Adam, 15 epoch)*

| Aktivasi (DNN 784–256–128–24) | Val. acc epoch 1 | Val. acc akhir | Akurasi uji |
|---|---|---|---|
| sigmoid | 38,71% | 92,46% | 72,17% |
| tanh | 48,43% | 100,00% | 77,84% |
| relu | 46,07% | 99,71% | 74,82% |

Sigmoid paling lambat dan paling rendah (validasi akhir 92,46%), sesuai teori saturasi gradien. tanh (100,00%) dan ReLU (99,71%) hampir setara pada validasi, tetapi ketiganya hanya mencapai 72,17%–77,84% pada data uji. Ini menegaskan bahwa keterbatasan DNN bukan pada fungsi aktivasinya, melainkan pada hilangnya informasi spasial.

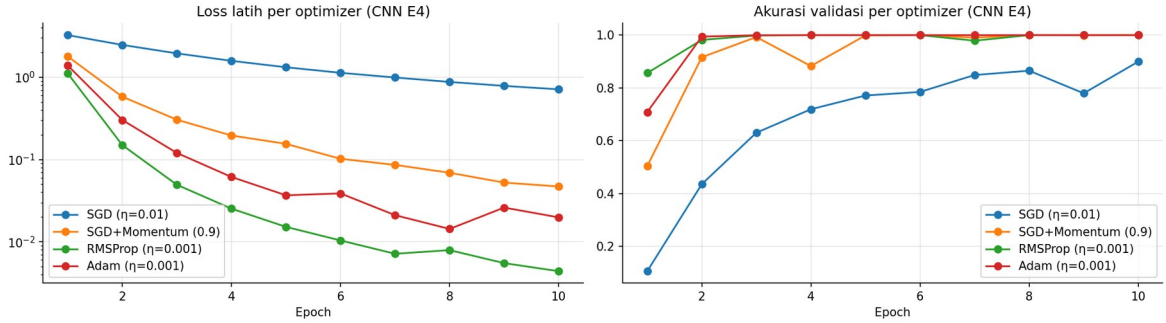



*Gambar 14 - Loss latih dan akurasi validasi per fungsi aktivasi*

*Tabel 17 - Perbandingan optimizer*

| Optimizer (CNN E4, 10 epoch) | Loss latih | Loss val. | Val. acc | Akurasi uji |
|---|---|---|---|---|
| SGD (η=0.01) | 0,7130 | 0,4467 | 89,95% | 83,76% |
| SGD+Momentum (0.9) | 0,0470 | 0,0016 | 100,00% | 98,12% |
| RMSProp (η=0.001) | 0,0044 | 0,0001 | 100,00% | 99,32% |
| Adam (η=0.001) | 0,0198 | 0,0001 | 100,00% | 99,11% |

SGD tanpa momentum tertinggal jauh setelah 10 epoch (validasi 89,95%). Menambahkan momentum 0,9 langsung membawa validasi ke 100% (Sutskever et al., 2013), dan optimizer adaptif RMSProp serta Adam mencapai loss validasi terendah (±0,0001).

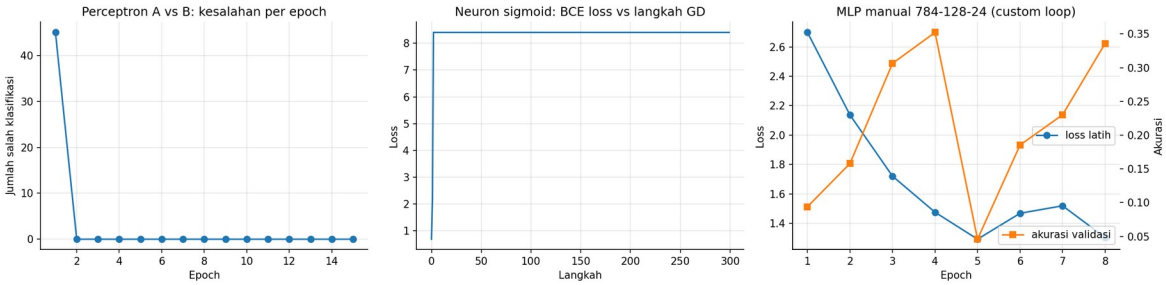



*Gambar 15 - Loss latih dan akurasi validasi per optimizer*

### *Fondasi: perceptron, neuron sigmoid, dan MLP dengan backpropagation manual*

Perceptron Rosenblatt (Rosenblatt, 1958) untuk huruf A vs B membuat 45 kesalahan pada epoch pertama dan 0 kesalahan sejak epoch kedua, dengan akurasi uji 100,00% (763 citra A/B). Ini menunjukkan kedua huruf separable linear dan sesuai dengan teorema konvergensi perceptron.

Dua percobaan lainnya justru memperlihatkan pentingnya pemilihan learning rate. Neuron sigmoid dengan gradient descent penuh (η = 0,5) divergen: loss naik dari 0,69 menjadi 8,40 pada langkah-langkah awal lalu terjebak, dan akurasi ujinya hanya 56,62%. Karena input tidak di-zero-center (nilai piksel rata-rata ±0,6 pada 784 dimensi), satu langkah besar membuat nilai z = w·x sangat besar sehingga sigmoid jenuh (saturasi). Gradiennya lalu mendekati nol dan bobot tidak lagi bergerak. MLP manual 784–128–24 dengan SGD η = 0,1 juga tidak stabil: akurasi validasinya naik-turun antara 4,6% dan 35,2%, dengan akurasi uji akhir 30,06%. Sebagai pembanding, MLP Keras E1 dengan η = 0,01 mencapai 67,99%. Pelajarannya: backpropagation berfungsi (loss MLP manual turun pada epoch awal), tetapi learning rate yang terlalu besar dan input yang tidak distandardisasi membuat optimisasi tidak stabil. Hal inilah yang diatasi pada E2–E5 dengan Adam, BatchNorm, dan scheduler.




*Gambar 16 - Kesalahan perceptron, loss neuron sigmoid, dan kurva MLP manual*

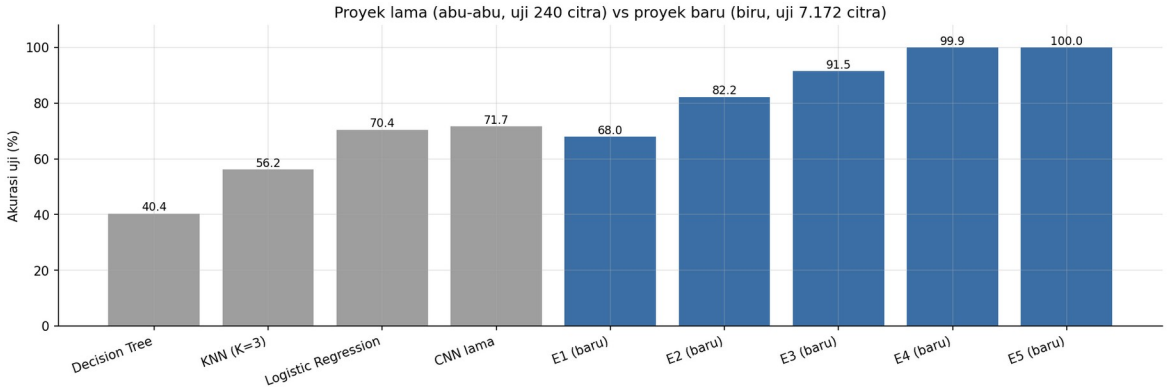
### *Dengan dan tanpa fitur unik, serta perbandingan dengan proyek sebelumnya*

*Tabel 18 - Ringkasan perbandingan tanpa dan dengan fitur unik*

| Perbandingan | Tanpa | Dengan |
|---|---|---|

| | | |
|---|---|---|
| Fitur 1: akurasi prediksi yang diterima (uji) | 100,00% | 100,00% (coverage 100,00%) |
| Fitur 1: input OOD yang ditolak (rata-rata 4 set) | 0% | 5,8% |
| Fitur 2: ukuran model | 5.274,3 KB | 446,0 KB (Dynamic range INT8) |
| Fitur 2: akurasi uji | 100,00% | 100,00% |
| Fitur 2: latensi per citra (tunggal) | 19,52 ms | 0,652 ms |

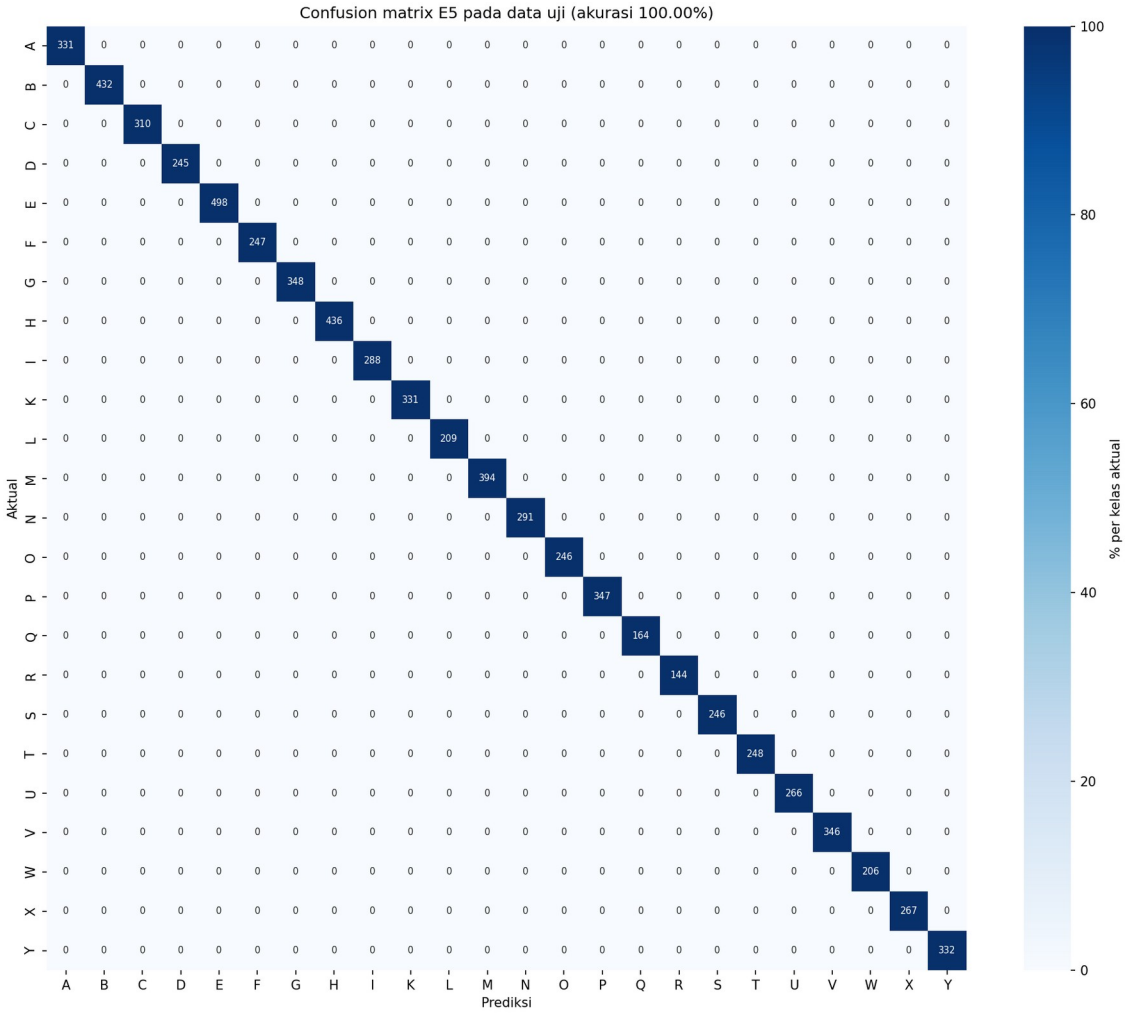
Gambar 17 membandingkan hasil proyek sebelumnya dengan proyek ini. Proyek sebelumnya mencapai 71,67% pada 240 citra, sedangkan proyek ini diuji pada 7.172 citra. Karena set ujinya berbeda, perbandingan ini bersifat indikatif, tetapi set uji baru 30× lebih besar dan sepenuhnya terpisah dari pelatihan.



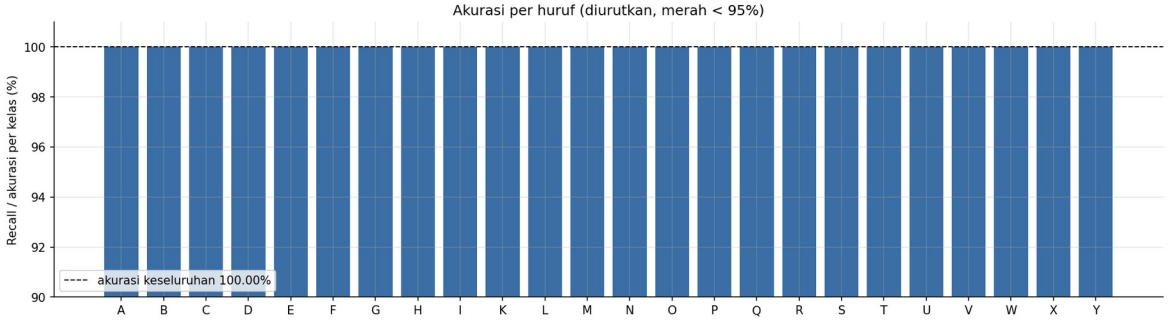

*Gambar 17 - Akurasi proyek sebelumnya (abu-abu) vs proyek ini (biru)*

**e. Analisis hasil**

Confusion matrix model E5 (Gambar 18) sepenuhnya diagonal: tidak ada satu pun pasangan huruf yang tertukar, dan akurasi setiap huruf 100% (Gambar 19). Pasangan yang sebelumnya sulit, seperti M→N, D→U, I→Y, S→M, dan V→U (lima kesalahan terbanyak pada CNN proyek sebelumnya), kini dapat dibedakan. Kesalahan yang masih tersisa pada skenario lain berkurang secara bertahap: ±2.296 citra salah pada E1, ±1.278 pada E2, ±610 pada E3, ±4 pada E4, dan 0 pada E5.



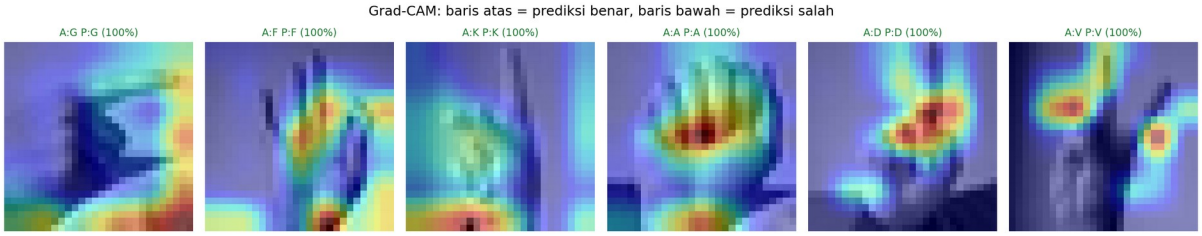

*Gambar 18 - Confusion matrix 24×24 model E5 pada data uji*




*Gambar 19 - Akurasi (recall) per huruf model E5*

Karena tidak ada kesalahan di data uji, analisis kegagalan dilakukan dari dua sisi lain. Pertama, Grad-CAM (Selvaraju et al., 2017) pada Gambar 20 memperlihatkan area yang paling memengaruhi keputusan. Pada huruf D dan V, heatmap terpusat pada jari, dan pada A pada kepalan tangan, sesuai harapan. Namun, pada F, K, dan G aktivasi terkuat justru berada di pergelangan tangan, tepi bawah citra, atau latar di sisi kanan. Artinya, model ikut memanfaatkan

petunjuk di luar bentuk jari (posisi pergelangan dan latar) yang kebetulan konsisten di dataset ini, atau shortcut. Petunjuk semacam ini berisiko tidak berlaku pada foto dunia nyata.




*Gambar 20 - Grad-CAM model E5 pada enam citra uji (semua prediksi benar)*

Kedua, input di luar domain menunjukkan bahwa model terlalu yakin (overconfident): derau acak diprediksi sebagai huruf P dengan keyakinan ±100% (Subbab 5.e). Temuan Grad-CAM dan OOD ini konsisten satu sama lain, dan menjadi dasar saran perbaikan: augmentasi latar/oklusi yang lebih kuat, data foto tangan dunia nyata, dan kalibrasi confidence.

Indikasi overfitting/underfitting per skenario:

- E1: latih 88,05%, validasi 87,87%, uji 67,99%, sehingga underfitting (akurasi latih dan validasi belum tinggi) sekaligus generalisasi lemah ke data uji.
- E2: latih 99,86%, validasi 100,00%, uji 82,18%, sehingga overfitting terhadap distribusi data latih: validasi hampir sempurna, tetapi akurasi uji jauh lebih rendah.
- E3: latih 100,00%, validasi 100,00%, uji 91,49%, sehingga overfitting terhadap distribusi data latih: validasi hampir sempurna, tetapi akurasi uji jauh lebih rendah.
- E4: latih 99,97%, validasi 100,00%, uji 99,94%, sehingga tidak ada indikasi overfitting maupun underfitting.
- E5: latih 99,68%, validasi 100,00%, uji 100,00%, sehingga tidak ada indikasi overfitting maupun underfitting.

## 7. Teknologi yang Digunakan

### a. Bahasa pemrograman

Python 3.13.16 (lingkungan Google Colab).

### b. Library dan versinya

*Tabel 19 - Library yang digunakan*

| Library | Versi | Kegunaan |
|---|---|---|
| TensorFlow | 2.21.0 | Tensor, tf.data, GradientTape, konverter TFLite |
| Keras | 3.13.2 | Model DNN/CNN, callback, augmentasi |
| NumPy | 2.4.6 | Komputasi numerik |
| scikit-learn | 1.6.1 | Split stratified, metrik, AUROC |
| OpenCV | 4.14.0 | Resize heatmap Grad-CAM |
| pandas, Matplotlib, Seaborn | bawaan Colab | Membaca CSV, visualisasi |

| | | |
|---|---|---|
| Pillow | bawaan | Praproses foto pada aplikasi |
| ai-edge-litert (LiteRT) | terbaru (PyPI) | Runtime TFLite pada aplikasi |
| Gradio | 5.9.1 | Antarmuka aplikasi web |
| python-docx | terbaru (PyPI) | Pembuatan laporan |

### c. Lingkungan pengembangan

Google Colaboratory dengan GPU Tesla T4. Seluruh notebook (training lima skenario, dua studi pendahuluan, evaluasi, fitur unik, dan persiapan deployment) selesai dalam 12,2 menit dengan seed global 42.

### d. Framework aplikasi dan platform hosting

- Gradio sebagai framework antarmuka web (Abid et al., 2019).
- Hugging Face Spaces (SDK Gradio, CPU gratis) sebagai platform hosting aplikasi.
- GitHub sebagai repositori kode yang berisi notebook, folder deploy, hasil, dan README.
- TensorFlow Lite (LiteRT) sebagai format dan runtime model pada aplikasi.

## 8. Deployment dan Demo

### a. Link deployment

*Tabel 20 - Link deployment*

| Artefak | Link |
|---|---|
| Aplikasi (Hugging Face Spaces) | https://huggingface.co/spaces/superstev/asl-sign-mnist |
| Repositori GitHub | https://github.com/stevbell123/asl-sign-mnist |
| Video demo & deployment | **[isi manual]** link video (YouTube/Google Drive) |

### b. Tahapan deployment

#### *Langkah 1: Menyimpan model*

Model akhir disimpan sebagai .keras (5.274 KB), lalu dikonversi ke model.tflite varian Dynamic range INT8 (446,0 KB). Daftar label dan ambang τ = 0,50 disimpan di config.json.

```
final_model.save("outputs/models/E5_model_akhir.keras")
final_model.export(export_dir)
conv = tf.lite.TFLiteConverter.from_saved_model(export_dir)
conv.optimizations = [tf.lite.Optimize.DEFAULT]          # dynamic range INT8
open("deploy/model.tflite", "wb").write(conv.convert())
```

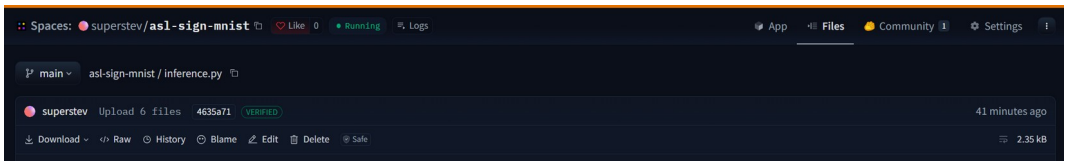



*Gambar 21 - Tangkapan layar langkah 1: menyimpan model*

### *Langkah 2: Menyiapkan skrip inferensi*

inference.py memuat model.tflite dengan Interpreter LiteRT, menyamakan praproses dengan pelatihan (grayscale → crop tengah → 28×28 → /255), lalu mengembalikan label, confidence, status diterima/ditolak, dan top-3. Pengujian di notebook menunjukkan prediksi skrip ini 100,00% sama dengan model Keras pada 1.000 citra uji (akurasi 100,00%).

```
clf = SignClassifier()            # memuat model.tflite + config.json
hasil = clf.predict(foto, threshold=0.5)
# -> {"label": "P", "confidence": 1.0, "accepted": True, "top3": [...]}
```

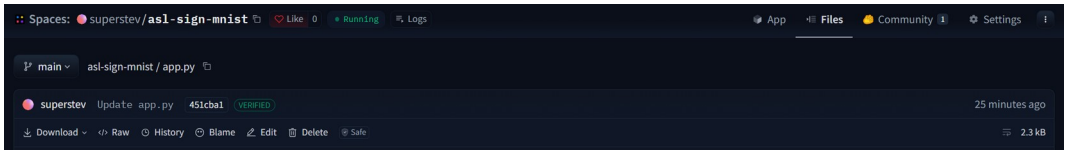



*Gambar 22 - Tangkapan layar langkah 2: menyiapkan skrip inferensi*

### *Langkah 3: Membangun antarmuka*

app.py menyediakan unggah foto, kamera, dan clipboard; slider ambang τ; Top-3 prediksi; status diterima/ditolak; pratinjau 28×28; dan contoh input.

```
with gr.Blocks() as demo:
    inp = gr.Image(type="numpy", sources=["upload", "webcam", "clipboard"])
    thr = gr.Slider(0, 1, value=clf.threshold, label="Ambang confidence")
    btn.click(prediksi, [inp, thr], [out_label, out_status, out_prev])
```

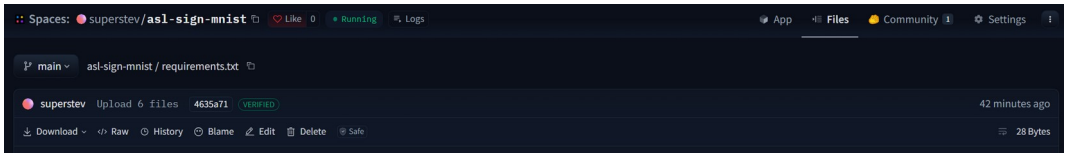



*Gambar 23 - Tangkapan layar langkah 3: membangun antarmuka*

### *Langkah 4: Mencatat dependensi*

requirements.txt hanya berisi ai-edge-litert, numpy, dan pillow, karena Gradio sudah disediakan oleh SDK Space. README.md berisi konfigurasi Space (sdk: gradio, app_file: app.py).

```
# requirements.txt
ai-edge-litert
numpy
pillow
```

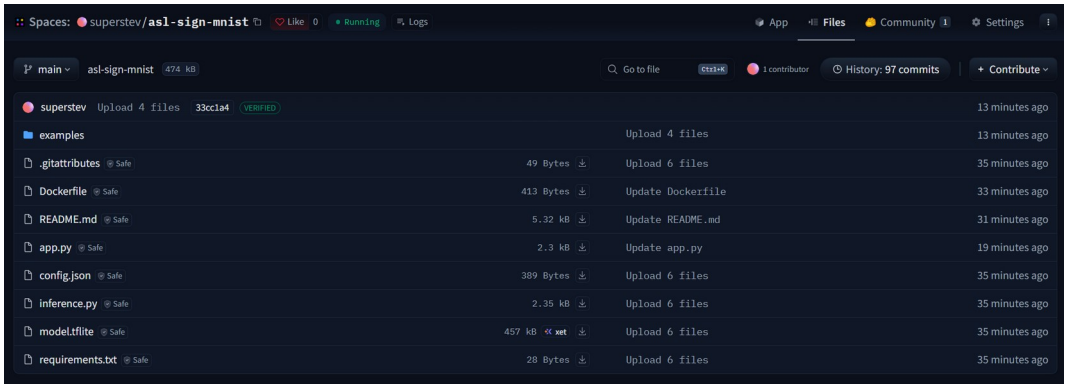



*Gambar 24 - Tangkapan layar langkah 4: mencatat dependensi*

### *Langkah 5: Mengunggah ke platform hosting*

Buat Space baru di huggingface.co (SDK Gradio, hardware CPU basic), lalu unggah seluruh isi folder deploy/ melalui menu Files atau git push. Kode lengkap juga di-push ke repositori GitHub.

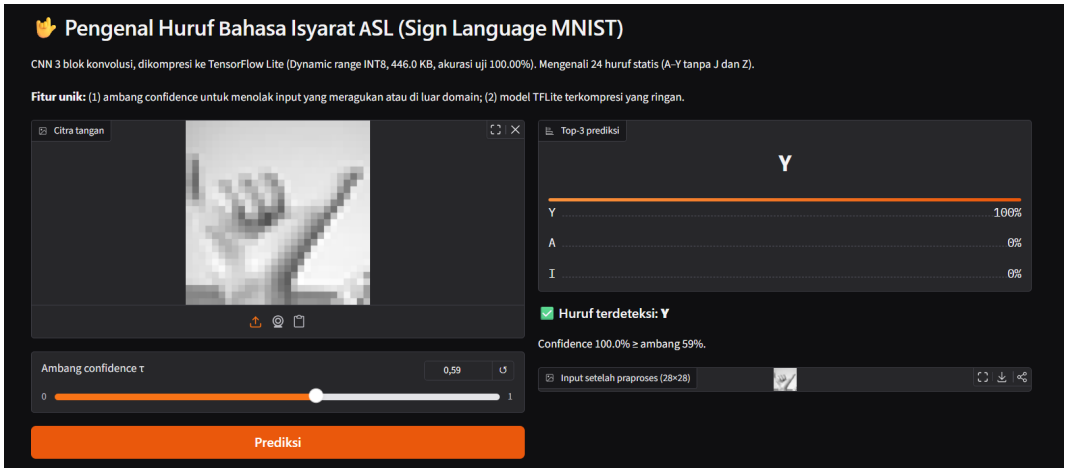
```
git clone https://huggingface.co/spaces/superstev/asl-sign-mnist
cp -r outputs/deploy/* asl-sign-mnist/
cd asl-sign-mnist && git add . && git commit -m "deploy" && git push
```




*Gambar 25 - Tangkapan layar langkah 5: mengunggah ke platform hosting*

### *Langkah 6: Menguji aplikasi*

Setelah build selesai (status Running), aplikasi diuji langsung melalui link pada Bagian 8.a dengan contoh dari data uji, contoh derau (input di luar domain), dan foto tangan asli. Contoh huruf Y diprediksi Y dengan confidence 100% dan status "Huruf terdeteksi" pada ambang 0,59, sehingga aplikasi berjalan sesuai hasil di notebook.



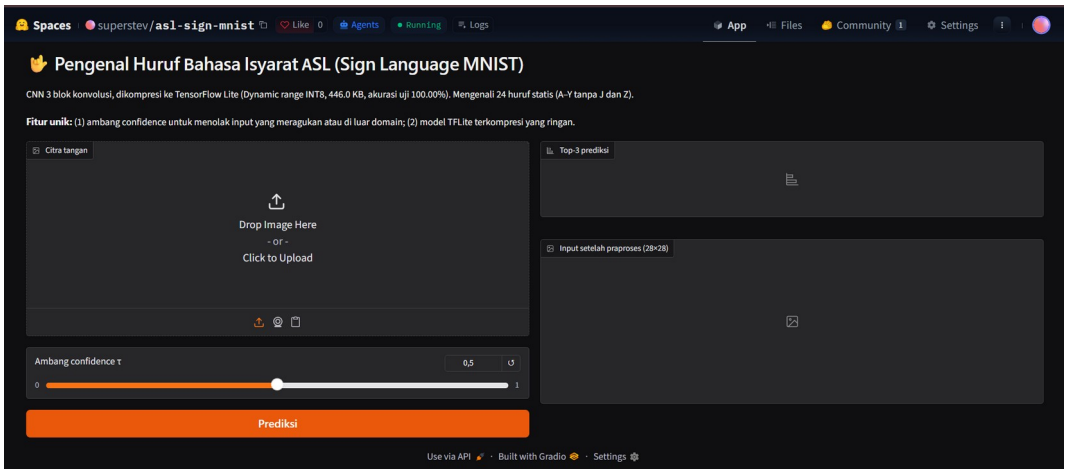

*Gambar 26 - Tangkapan layar langkah 6: menguji aplikasi*

## c. Penjelasan video demo dan deployment

### *i. Tampilan user interface*

Antarmuka terdiri atas dua kolom. Kolom kiri berisi input citra (unggah, kamera, clipboard), slider ambang τ, dan tombol Prediksi. Kolom kanan berisi Top-3 prediksi beserta

probabilitasnya, status keputusan (✅ diterima atau ⚠️ ditolak), dan pratinjau citra 28×28 setelah praproses. Di bawahnya tersedia contoh input yang dapat diklik.




*Gambar 27 - Tampilan antarmuka aplikasi di Hugging Face Spaces*

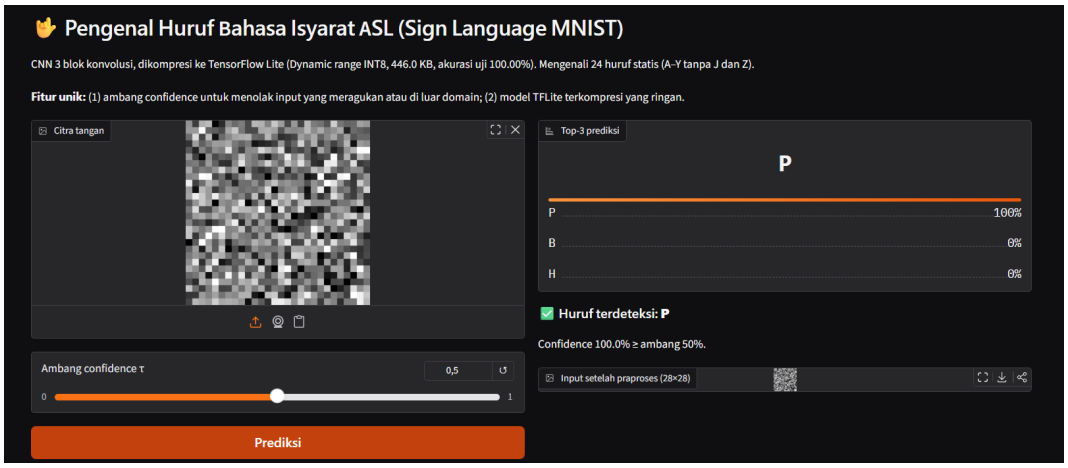
### *ii. Contoh input dan output*

Gambar 28 dan Tabel 21 memperlihatkan output aplikasi (melalui inference.py yang sama) untuk contoh input. Ketiga contoh dari data uji diprediksi benar dengan keyakinan ±100%. Karena model tidak membuat kesalahan pada data uji, contoh prediksi salah diambil dari input di luar domain: derau acak diprediksi sebagai P dengan keyakinan 100% dan tidak tertolak. Untuk video, contoh salah tambahan sebaiknya memakai foto tangan asli dengan latar ramai atau pencahayaan berbeda.




*Gambar 28 - Contoh input dan output aplikasi (hijau = benar, merah = salah)*

Hal yang sama terlihat di aplikasi yang sudah di-deploy (Gambar 29): citra derau dari contoh diterima sebagai huruf P dengan confidence 100% pada ambang 0,5. Ini contoh prediksi salah yang ditampilkan di video demo.




*Gambar 29 - Tangkapan layar aplikasi: input derau diterima sebagai huruf P (contoh prediksi salah)*

*Tabel 21 - Ringkasan contoh input dan output aplikasi*

| Berkas contoh | Label aktual | Output aplikasi | Confidence | Keputusan |
|---|---|---|---|---|
| 1_benar_aktual_W.png | W | W | 100,0% | Diterima |
| 2_benar_aktual_P.png | P | P | 100,0% | Diterima |
| 3_benar_aktual_F.png | F | F | 100,0% | Diterima |
| 4_ood_derau.png | (OOD) | P | 100,0% | Diterima (salah) |

### *iii. Demonstrasi dua fitur unik*

- Fitur 1: unggah citra polos atau foto yang bukan tangan, lalu geser slider τ dari 0,5 ke ±0,9 untuk memperlihatkan bahwa keputusan berubah dari diterima menjadi ditolak. Jelaskan juga secara jujur bahwa derau acak tetap lolos karena model terlalu yakin (keterbatasan yang dibahas di Subbab 5.e).
- Fitur 2: tunjukkan berkas model.tflite (446,0 KB) di tab Files Space, bandingkan dengan model Keras (5.274 KB), dan tunjukkan bahwa aplikasi berjalan tanpa TensorFlow penuh dengan respons cepat.

### *iv. Penjelasan singkat tahapan deployment*

Video menjelaskan alurnya secara singkat: model disimpan dan dikonversi ke TFLite, skrip inferensi dan antarmuka Gradio disiapkan, dependensi dicatat, berkas diunggah ke Hugging Face Spaces, lalu aplikasi diuji. Pembagian segmen video ada pada Tabel 22.

*Tabel 22 - Pembagian segmen video (setiap anggota menjelaskan bagiannya)*

| Segmen | Materi | Pembawa |
|---|---|---|
| 1 | Pembuka, masalah, dataset, dan arsitektur CNN | Steven David H Malau |
| 2 | EDA, preprocessing, dan pipeline tf.data | Wan Saka Nasa |
| 3 | DNN, perceptron, studi aktivasi dan optimizer | Edwin Valentino Tarigan |
| 4 | Hasil evaluasi, Grad-CAM, dan dua fitur unik | Eric Chandra |
| 5 | Demo aplikasi dan tahapan deployment | Dhaifan Al Ghani |

## 9. Lampiran

### a. Cuplikan kode penting

#### *Preprocessing dan pipeline tf.data*

```
X_full = (X_all_raw.astype("float32") / 255.0)[..., np.newaxis]     # (N,28,28,1)
X_train, X_val, y_train, y_val = train_test_split(
    X_full, y_full, test_size=0.10, stratify=y_full, random_state=SEED)

geo_augment = keras.Sequential([layers.RandomRotation(0.03, fill_mode="nearest"),
                                layers.RandomZoom(0.10, fill_mode="nearest"),
                                layers.RandomTranslation(0.10, 0.10,
fill_mode="nearest")])
def photometric_augment(x, y):
    x = tf.image.random_brightness(x, max_delta=0.10)
    x = tf.image.random_contrast(x, lower=0.8, upper=1.2)
    return tf.clip_by_value(x, 0.0, 1.0), y
```

#### *Arsitektur CNN akhir*

```
def conv_bn(filters, name):
    return [layers.Conv2D(filters, 3, padding="same", use_bias=False,
                          kernel_initializer="he_normal", name=f"{name}_conv"),
            layers.BatchNormalization(name=f"{name}_bn"), layers.Activation("relu",
name=f"{name}_relu")]

model = keras.Sequential(
    [keras.Input((28, 28, 1))]
    + conv_bn(32, "b1c1") + conv_bn(32, "b1c2") + [layers.MaxPooling2D(2),
layers.Dropout(0.25)]
    + conv_bn(64, "b2c1") + conv_bn(64, "b2c2") + [layers.MaxPooling2D(2),
layers.Dropout(0.25)]
    + conv_bn(128, "b3c1") + [layers.MaxPooling2D(2), layers.Dropout(0.30)]
    + [layers.Flatten(), layers.Dense(256, use_bias=False), layers.BatchNormalization(),
       layers.Activation("relu"), layers.Dropout(0.4), layers.Dense(24,
activation="softmax")])
model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="sparse_categorical_crossentropy", metrics=["accuracy"])
callbacks = [EarlyStopping(monitor="val_loss", patience=10, restore_best_weights=True),
             ModelCheckpoint("E5_best.weights.h5", save_best_only=True,
save_weights_only=True),
             ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=3, min_lr=1e-5)]
```

#### *Fitur unik 1: penentuan ambang τ*

```
val_aug_ds = make_dataset(X_val, y_val, augment=True)   # validasi teraugmentasi (bukan
uji)
prob_va = final_model.predict(val_aug_ds, verbose=0)
conf_va, pred_va = prob_va.max(1), prob_va.argmax(1)
for t in np.arange(0.0, 1.0, 0.01):          # tau terkecil yang memenuhi target
    m = conf_va >= t
    if (pred_va[m] == y_val[m]).mean() >= TARGET_ACC_DITERIMA:
        TAU = float(t); break
TAU = max(TAU, 0.50)
```

#### *Fitur unik 2: konversi ke TensorFlow Lite*

```
final_model.export(export_dir)
```

```
def convert(varian):
    conv = tf.lite.TFLiteConverter.from_saved_model(export_dir)
    if varian == "FP16":
        conv.optimizations = [tf.lite.Optimize.DEFAULT]; conv.target_spec.supported_types
= [tf.float16]
    elif varian == "Dynamic range INT8":
        conv.optimizations = [tf.lite.Optimize.DEFAULT]
    elif varian == "Full INT8":
        conv.optimizations = [tf.lite.Optimize.DEFAULT]; conv.representative_dataset =
representative_data
    return conv.convert()
```

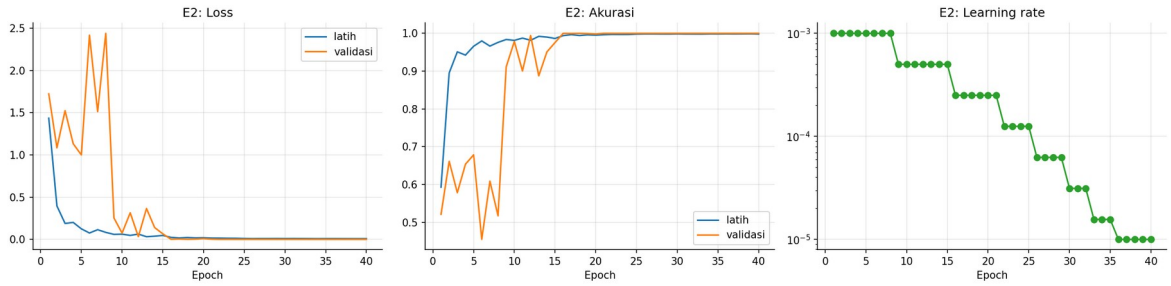
### *Inferensi pada aplikasi (inference.py)*

```
def preprocess(image):
    img = Image.fromarray(np.asarray(image).astype(np.uint8)).convert("L")
    w, h = img.size; s = min(w, h)
    img = img.crop(((w - s) // 2, (h - s) // 2, (w - s) // 2 + s, (h - s) // 2 + s))
    img = img.resize((28, 28), Image.BOX)
    return (np.asarray(img, dtype=np.float32) / 255.0).reshape(1, 28, 28, 1)

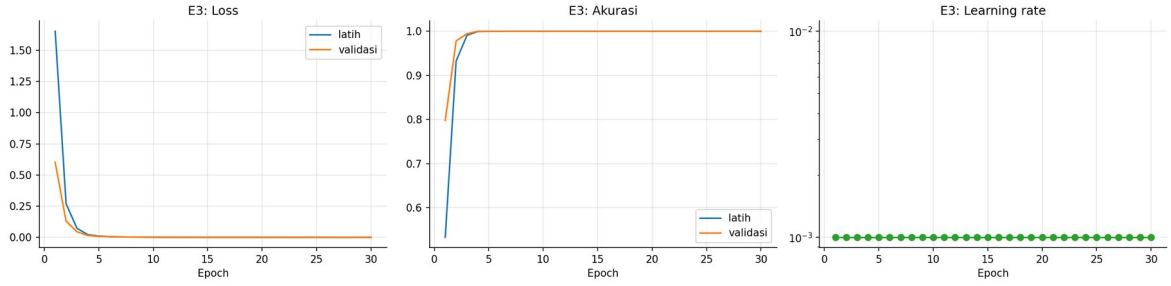
def predict(self, image, threshold=None):
    self.itp.set_tensor(self.inp["index"], self.preprocess(image)); self.itp.invoke()
    p = self.itp.get_tensor(self.out["index"])[0]
    tau = self.threshold if threshold is None else threshold
    return {"label": self.labels[p.argmax()], "confidence": float(p.max()),
            "accepted": bool(p.max() >= tau)}
```

## b. Grafik evaluasi

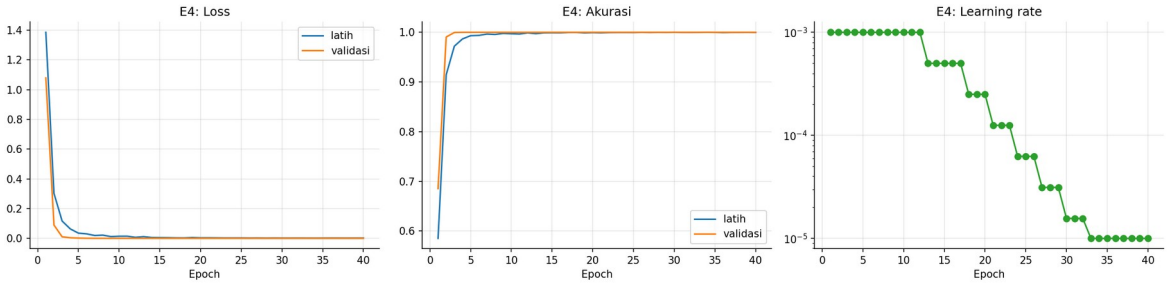
Grafik evaluasi tambahan yang melengkapi Bagian 6:



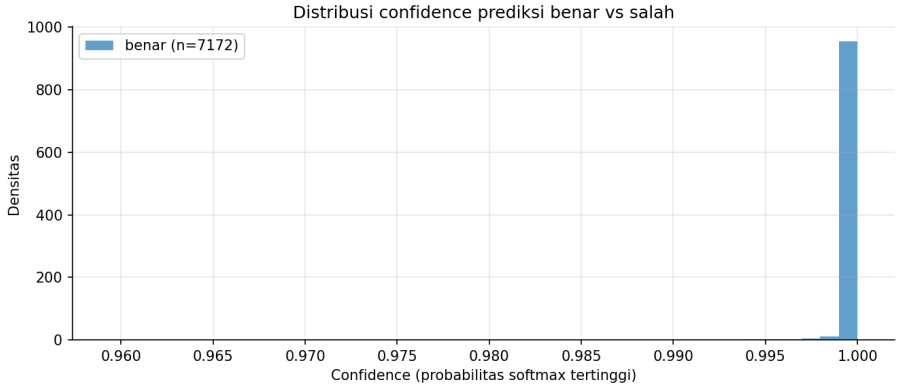

*Gambar 30 - Kurva loss dan akurasi (latih vs validasi) DNN teregularisasi E2*



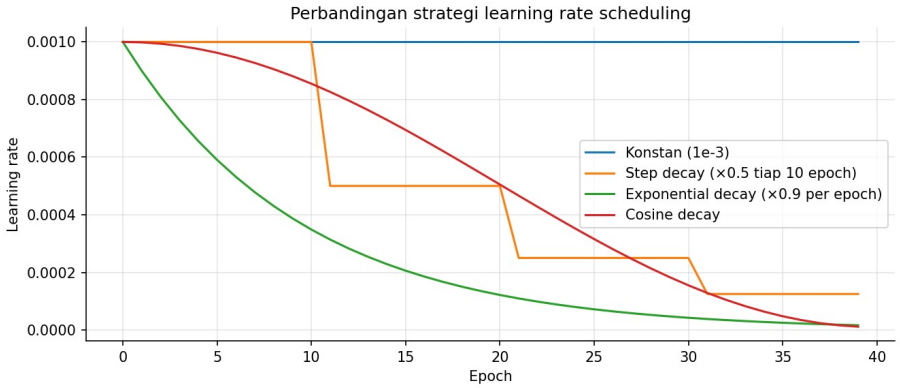

*Gambar 31 - Kurva loss dan akurasi (latih vs validasi) CNN baseline E3*



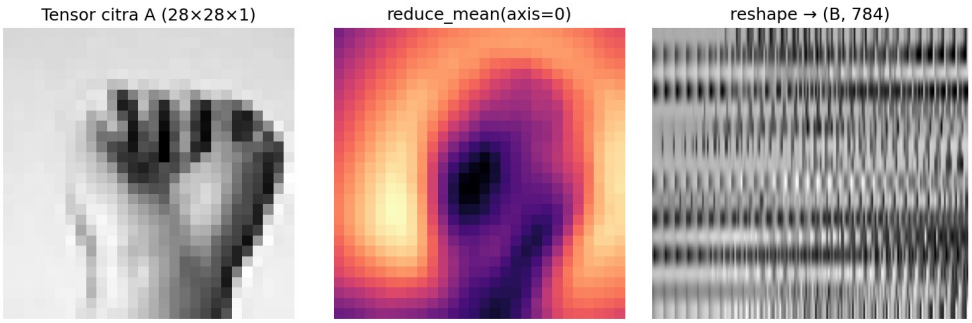

*Gambar 32 - Kurva loss dan akurasi (latih vs validasi) CNN teregularisasi E4*




*Gambar 33 - Distribusi confidence prediksi model E5 pada data uji (seluruhnya benar)*




*Gambar 34 - Strategi learning rate scheduling (konsep)*




*Gambar 35 - Representasi tensor: citra (28×28×1), reduce_mean, dan reshape ke (B, 784)*

## 10. Daftar Pustaka


Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., … Zheng, X. (2016). TensorFlow: A system for large-scale machine learning. In 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16) (pp. 265–283).

Abid, A., Abdalla, A., Abid, A., Khan, D., Alfozan, A., & Zou, J. (2019). Gradio: Hassle-free sharing and testing of ML models in the wild. arXiv:1906.02569.

Chollet, F., et al. (2015). Keras. https://keras.io

Datamunge. (2017). Sign Language MNIST: Drop-in replacement for MNIST for hand gesture recognition tasks [Dataset]. Kaggle. https://www.kaggle.com/datasets/datamunge/sign-language-mnist

Gad, A. F., & Jarmouni, F. E. (2020). Introduction to deep learning and neural networks with Python™: A practical guide. Academic Press.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.

He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In Proceedings of the IEEE International Conference on Computer Vision (pp. 1026–1034).

Hendrycks, D., & Gimpel, K. (2017). A baseline for detecting misclassified and out-of-distribution examples in neural networks. In International Conference on Learning Representations (ICLR).

Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In Proceedings of the 32nd International Conference on Machine Learning (pp. 448–456).

Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., & Kalenichenko, D. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (pp. 2704–2713).

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In International Conference on Learning Representations (ICLR).

Kneusel, R. T. (2021). Practical deep learning: A Python-based introduction. No Starch Press.

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), 2278–2324.

Nair, V., & Hinton, G. E. (2010). Rectified linear units improve restricted Boltzmann machines. In Proceedings of the 27th International Conference on Machine Learning (pp. 807–814).

Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. Psychological Review, 65(6), 386–408.

Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. In Proceedings of the IEEE International Conference on Computer Vision (pp. 618–626).

Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on image data augmentation for deep learning. Journal of Big Data, 6(1), 60.

Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. In International Conference on Learning Representations (ICLR).

Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. Information Processing & Management, 45(4), 427–437.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. Journal of Machine Learning Research, 15(56), 1929–1958.

Sutskever, I., Martens, J., Dahl, G., & Hinton, G. (2013). On the importance of initialization and momentum in deep learning. In Proceedings of the 30th International Conference on Machine Learning (pp. 1139–1147).

World Health Organization. (2021). World report on hearing. WHO.

Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). Dive into deep learning. Cambridge University Press. https://d2l.ai

## BAGIAN B. TAUTAN

Semua tautan berikut dapat dibuka tanpa meminta akses. Laporan, source code, dan video membahas implementasi yang sama, dengan dataset, arsitektur, dan angka hasil evaluasi yang konsisten.

1. **Link source code (GitHub):** https://github.com/stevbell123/asl-sign-mnist
2. **Link video demo beserta tahapan deployment:** **[isi manual]** link YouTube/Google Drive (akses publik)
3. **Link aplikasi yang sudah di-deploy:** https://huggingface.co/spaces/superstev/asl-sign-mnist